Microsoft Fabric for Data Engineers

The complete practitioner's study guide: OneLake, Lakehouse, Spark, Warehouse, Data Factory, Real-Time Intelligence, security, CI/CD and performance

Written from the seat of a data engineer with 15 years of building pipelines, warehouses and lakehouses on SQL Server, Hadoop, Synapse, Databricks and Fabric.

Scope: the whole Fabric platform for data engineering work. Aligned with, but broader than, exam DP-700 (Fabric Data Engineer Associate).

Current as of October 2026 (Fabric September 2026 feature summary, FabCon Europe 2026, Fabric Runtime 2.0 GA)

Contents

Table of Contents

- **Fabric in 2026: scope and orientation**
  - How to use this guide
  - What changed recently (status as of October 2026)
  - Fabric workloads at a glance
- **Fabric architecture and OneLake**
  - The SaaS model
  - OneLake
  - Shortcuts
  - Mirroring (overview)
  - OneLake catalog
  - OneLake storage features
- **Workspaces, capacities and licensing**
  - Workspaces
  - Capacities and F-SKUs
- **Lakehouse**
  - Lakehouse structure
  - Medallion architecture
  - Lakehouse utilities
- **Spark and notebooks**
  - Spark compute
  - Notebooks
  - PySpark essentials
- **Delta Lake operations**
  - Writing and changing data
  - Table layout and maintenance
  - History and recovery
  - Schema management
  - Slowly changing dimensions
- **Data Warehouse**
  - Warehouse architecture
  - T-SQL DDL
  - Loading data
  - Querying across items
  - Security in the warehouse
  - Statistics, recovery and snapshots
  - Monitoring the warehouse
- **Data Factory: pipelines, Copy job and Dataflows Gen2**
  - Choosing an ingestion tool
  - Pipelines
  - Copy job
  - Dataflows Gen2
- **Real-Time Intelligence**
  - RTI building blocks
  - Eventstreams
  - Eventhouse and KQL databases
  - Real-Time dashboards and Activator
- **Power BI semantic models and Direct Lake**
  - Storage modes
  - DAX basics for engineers
- **SQL database in Fabric and mirroring**
  - SQL database in Fabric
  - Mirrored databases
- **Security and governance**
  - Access model layers
  - Workspace roles
  - Item permissions
  - OneLake security
  - Information protection and Purview
  - Network security
- **CI/CD and automation**
  - Git integration
  - Deployment pipelines
  - fabric-cicd and code-first deployment
- **Monitoring and operations**
  - Monitoring hub
- **Performance tuning**
  - Spark and Delta tuning
  - Warehouse and SQL tuning
  - KQL and capacity tuning
- **Fabric compared with other platforms**
  - Platform comparison
- **Interview Q&A: Fabric data engineer**
  - Fundamentals
  - Design and scenarios
  - Troubleshooting and performance
  - Experience and judgement
- **Cheat sheet**
  - Platform map
  - OneLake paths
  - Capacity
  - Workspace roles
  - Item permissions
  - Runtimes
  - Spark settings
  - notebookutils
  - PySpark I/O
  - Delta commands
  - SCD quick rules
  - Materialized lake view
  - Warehouse T-SQL
  - Warehouse security
  - Pipeline expressions
  - Watermark pattern
  - Ingestion choice
  - KQL core
  - KQL management
  - Direct Lake
  - OneLake security
  - Network
  - CI/CD
  - Monitoring
  - Perf one-liners
  - Limits & defaults

Fabric in 2026: scope and orientation

How to use this guide

Each chapter follows the same pattern. A `##` heading is a subject area, a `###` heading is a concept you will be asked about, and each concept has a short explanation, the syntax, and a worked example you can paste into a notebook, SQL editor, KQL queryset or terminal. Orange boxes flag traps that show up in production and in interviews. The last pages hold 40 interview questions and a one-page cheat sheet.

The examples share one fictional company, **Contoso Retail**, with these items:

Item	Type	Purpose
`ws-retail-dev` / `-test` / `-prod`	Workspaces	Lifecycle stages on capacity `cap-retail-f64`
`lh_bronze`, `lh_silver`, `lh_gold`	Lakehouses (schema-enabled)	Medallion layers
`wh_sales`	Warehouse	Star schema served to Power BI
`eh_telemetry` / `kql_iot`	Eventhouse / KQL database	Store sensor and clickstream events
`pl_daily_sales`	Pipeline	Orchestration of the nightly load
`sm_sales`	Semantic model (Direct Lake)	Reporting

```
Tenant (Entra ID)
 └─ Capacity  cap-retail-f64 (F64, West Europe)
     ├─ Workspace ws-retail-prod  (domain: Sales)
     │    ├─ lh_bronze  lh_silver  lh_gold   (Lakehouse + SQL analytics endpoint)
     │    ├─ wh_sales                        (Warehouse)
     │    ├─ pl_daily_sales  cj_crm_copy  df_customers  (Pipeline, Copy job, Dataflow Gen2)
     │    ├─ nb_silver_orders  sjd_streaming (Notebook, Spark job definition)
     │    ├─ eh_telemetry → kql_iot, es_clicks, act_alerts (RTI items)
     │    └─ sm_sales, rpt_sales             (Semantic model, report)
     └─ OneLake: one logical lake per tenant, abfss://<workspace>@onelake.dfs.fabric.microsoft.com/
```

What changed recently (status as of October 2026)

Fabric ships monthly. The table lists the changes a data engineer should know about, with the status reported in Microsoft's feature summaries and FabCon coverage. Preview features change, so check the release notes before relying on one in production.

Feature	Status (late 2026)	Why it matters
Fabric Runtime 2.0 (Spark 4.1, Delta 4.2, Python 3.13, Java 21, Scala 2.13)	GA; planned as the default for new workspaces from late Sept 2026	Python 3.13 can break old wheels; retest environments
Native execution engine	GA on Runtime 1.3 and 2.0	Vectorized C++ execution for Spark SQL/DataFrames, no code change
Materialized lake views (Spark SQL)	GA (March 2026); PySpark authoring in preview	Declarative medallion layers with managed refresh and data quality
OneLake security (table/folder/row/column roles)	GA in spring 2026 (sources say April or May)	One permission model enforced across Spark, SQL endpoint and Direct Lake
SQL database in Fabric	GA (Ignite, Nov 2025)	OLTP database that auto-replicates to OneLake
Copy job (incremental, CDC, SCD Type 2 option)	GA; CDC and SCD2 features announced 2026	Simplest ingestion path without a pipeline

Feature	Status (late 2026)	Why it matters
Custom live pools (Spark)	GA (Sept 2026)	Pre-warmed custom pools with fast session start
Warehouse custom SQL pools, distributed bitmap filters	GA (Sept 2026)	Workload isolation and faster star joins
Warehouse snapshots	GA	Stable read-only point-in-time view for reporting during ETL
fabric-cicd Python library	Officially supported by Microsoft (Feb 2026)	Code-first deployment across workspaces
Eventstream SQL operator	GA (2026)	Code-first stream processing with multiple destinations
Mirroring for SAP, Google BigQuery, SharePoint lists	GA (2026); Amazon S3 mirroring announced	Zero-ETL replication from more sources
Data engineering agent (from the Osmos acquisition)	Preview (FabCon Europe 2026)	Agent that plans and runs migrations and ETL within guardrails
F0 SKU with on-demand billing	Announced Oct 2026; pricing not published	Pay-per-use entry point

Exam trap: older material says "MERGE is not supported in Fabric Warehouse" and "Runtime 1.3 is the latest". Both are out of date. MERGE is documented for Fabric Data Warehouse and Runtime 2.0 is GA. Interviewers who used Fabric in 2024 may still hold the older view; say what changed and when.

Fabric workloads at a glance

Workload / item	Engine	Language	Write path	Typical use
Lakehouse	Spark + SQL analytics endpoint	PySpark, Spark SQL, Scala, R; T-SQL read-only	Spark, pipelines, dataflows, shortcuts	Bronze/silver/gold, data science
Warehouse	Polaris distributed SQL	T-SQL (read/write)	T-SQL, COPY INTO, pipelines, dataflows	Enterprise star schemas, SQL-first teams
SQL database	SQL Database engine (OLTP)	T-SQL	Apps	Operational apps, metadata/control tables
Eventhouse / KQL database	Kusto	KQL, T-SQL subset	Eventstream, ingestion APIs	Logs, telemetry, time series
Pipeline / Copy job / Dataflow Gen2	Data Factory	Expressions, Power Query M	Any destination	Orchestration and ingestion
Semantic model	Analysis Services (VertiPaq)	DAX	Direct Lake, Import, DirectQuery	Reporting
Mirrored database	Replication into OneLake	none	Managed CDC	Zero-ETL copies of operational data

Fabric architecture and OneLake

The SaaS model

Tenant, capacity, workspace, item

Fabric is a SaaS platform. You do not provision clusters or storage accounts. A **tenant** maps to your Entra ID directory. A **capacity** is a pool of compute (capacity units, CU) bought as an F-SKU in an Azure region. A **workspace** is a container for items, assigned to one capacity. An **item** is a notebook, lakehouse, warehouse, pipeline, report and so on. Every item that stores tabular data writes it to OneLake as Delta Lake tables.

Syntax (REST, list items in a workspace):

```
GET https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}/items?type=Lakehouse
Authorization: Bearer <Entra token for https://api.fabric.microsoft.com/.default>
```

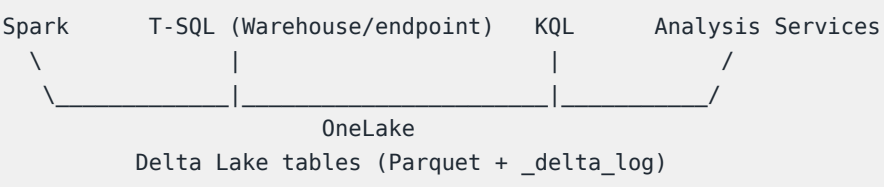
Example: list all lakehouses in the prod workspace from Python with the `azure-identity` library.

```python
import requests
from azure.identity import DefaultAzureCredential

token = DefaultAzureCredential().get_token("https://api.fabric.microsoft.com/.default").token
ws_id = "b2c1a9e0-1111-4c3e-9f00-2b6f0d1e7a10"   # ws-retail-prod
r = requests.get(
    f"https://api.fabric.microsoft.com/v1/workspaces/{ws_id}/items",
    params={"type": "Lakehouse"},
    headers={"Authorization": f"Bearer {token}"},
)
r.raise_for_status()
for item in r.json()["value"]:
    print(item["displayName"], item["id"])   # lh_bronze ..., lh_silver ..., lh_gold ...
```

Compute and storage separation

All engines read and write the same open format in OneLake, so storage is shared and compute is per engine. Spark writes a Delta table in a lakehouse; the SQL analytics endpoint, a semantic model in Direct Lake mode and a KQL shortcut can read it without a copy. Compute for all engines draws from the one capacity, which is why a heavy Spark job can slow down report queries in the same capacity.



Example: write with Spark, read with T-SQL, no copy.

```python
# Notebook attached to lh_silver
df = spark.read.table("lh_bronze.sales.orders_raw")
df.filter("order_status <> 'TEST'").write.mode("overwrite").saveAsTable("sales.orders")
```

```sql
-- SQL analytics endpoint of lh_silver, seconds later (metadata sync)
SELECT TOP 10 order_id, customer_id, order_total
FROM sales.orders
ORDER BY order_date DESC;
```

OneLake

OneLake fundamentals

OneLake is a single logical data lake per tenant, built on Azure Data Lake Storage Gen2. It is created automatically and exposes ADLS Gen2 APIs and the ABFS driver, so existing tools (Azure Storage Explorer, Databricks, azcopy, the Python ADLS SDK) work against it. Each workspace is a container, each item is a folder. Tables live under `Tables/`, unstructured data under `Files/`.

Syntax:

```
abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<item>.<itemtype>/Tables/<schema>/<table>
abfss://<workspaceId>@onelake.dfs.fabric.microsoft.com/<itemId>/Files/<path>
https://onelake.dfs.fabric.microsoft.com/<workspace>/<item>.Lakehouse/Files/...
```

Example: read a OneLake table from a notebook in another workspace and upload a file with azcopy.

```
path = ("abfss://ws-retail-prod@onelake.dfs.fabric.microsoft.com/"
        "lh_gold.Lakehouse/Tables/sales/fact_sales")
df = spark.read.format("delta").load(path)
df.groupBy("store_id").sum("net_amount").show(5)
```

```
azcopy login
azcopy copy "./exports/stores_2026-10-26.csv" \
  "https://onelake.blob.fabric.microsoft.com/ws-retail-
prod/lh_bronze.Lakehouse/Files/landing/stores/" \
  --trusted-microsoft-suffixes "fabric.microsoft.com"
```

> **Best practice:** use GUID-based paths (`<workspaceId>/<itemId>`) in production code. Display names change when someone renames a workspace; GUIDs do not.

Delta Lake and Parquet

Every Fabric table is a Delta Lake table: Parquet data files plus a `_delta_log` folder of JSON commits and Parquet checkpoints. The log gives ACID transactions, time travel and schema enforcement. Delta is the common language between engines. The Warehouse also writes Delta logs, so Spark can read warehouse tables through OneLake.

Component	Contents	Notes
`part-*.parquet`	Column data, compressed	V-Order applied by Fabric writers by default where enabled
`_delta_log/000...N.json`	Add/remove file actions, schema, metadata	One file per commit
`_delta_log/*.checkpoint.parquet`	Snapshot of the log	Every 10 commits by default
`deletion_vectors`	Soft-deleted row bitmaps	Avoids rewriting files on DELETE/UPDATE

Syntax:

```
DESCRIBE DETAIL sales.orders;       -- format, location, numFiles, sizeInBytes
DESCRIBE HISTORY sales.orders;      -- versions, operations, user
SHOW TBLPROPERTIES sales.orders;
```

Example: inspect a table to see whether it needs compaction.

```python
detail = spark.sql("DESCRIBE DETAIL lh_silver.sales.orders").collect()[0]
avg_mb = detail.sizeInBytes / max(detail.numFiles, 1) / 1024 / 1024
print(f"{detail.numFiles} files, average {avg_mb:.1f} MB")
if avg_mb < 64:
    spark.sql("OPTIMIZE lh_silver.sales.orders")   # compact small files
```

V-Order

V-Order is a write-time optimization applied to Parquet files (special sorting, row-group distribution, dictionary encoding and compression). Files stay 100% Parquet compliant, so any reader can open them, but Fabric engines (VertiPaq for Direct Lake, the SQL engine, Spark) read V-Ordered files faster. Writes cost roughly 15% more time. Recent workspaces default to resource profiles that leave V-Order off for write-heavy Spark work, so check the setting rather than assuming it.

Syntax:

```python
spark.conf.set("spark.sql.parquet.vorder.default", "true")        # session (Runtime 1.3+)
```

```sql
ALTER TABLE gold.fact_sales SET TBLPROPERTIES ('delta.parquet.vorder.enabled' = 'true');
OPTIMIZE gold.fact_sales VORDER;
```

Example: turn on V-Order only for the gold tables that feed Direct Lake.

```python
gold_tables = ["gold.fact_sales", "gold.dim_customer", "gold.dim_product"]
for t in gold_tables:
    spark.sql(f"ALTER TABLE lh_gold.{t} SET TBLPROPERTIES ('delta.parquet.vorder.enabled'='true')")
    spark.sql(f"OPTIMIZE lh_gold.{t} VORDER")   # rewrite existing files once
```

> **Trap:** V-Order helps reads, not writes. Use it on gold tables read by Power BI and SQL. On bronze tables written every few minutes, it adds cost for no benefit.

Resource profiles

Spark environments ship with resource profiles that set write and read defaults in one switch: `writeHeavy` (default for new workspaces; V-Order off, optimize write off), `readHeavyForSpark`, and `readHeavyForPBI` (V-Order on, optimize write on).

```python
spark.conf.set("spark.fabric.resourceProfile", "readHeavyForPBI")
```

Example: a gold-layer notebook switches profile before writing the serving tables.

```python
spark.conf.set("spark.fabric.resourceProfile", "readHeavyForPBI")
fact = spark.table("lh_silver.sales.orders_enriched")
fact.write.mode("overwrite").saveAsTable("lh_gold.gold.fact_sales")
```

Shortcuts

What shortcuts are

A shortcut is a pointer in OneLake to data stored elsewhere. It looks like a folder or table, but no data is copied. Readers authenticate to the target either with their own identity (OneLake-to-OneLake shortcuts) or with the connection's credential (external shortcuts). Shortcuts placed directly under `Tables/` must point at a Delta table to show up as tables; anything else goes under `Files/`.

Target	Auth	Caching	Gotcha
OneLake (another lakehouse, warehouse, KQL DB)	Caller's identity, passthrough	n/a	Caller needs access to the source item

Target	Auth	Caching	Gotcha
ADLS Gen2	Org account, SAS, service principal, workspace identity	No	Must point at DFS endpoint; trusted workspace access for firewalled accounts
Amazon S3 / S3-compatible	Access key	Shortcut cache (1-28 days)	Egress charged by AWS; cache reduces it
Google Cloud Storage	HMAC key	Shortcut cache	Same egress note
Dataverse	Org account	No	Created from Dataverse side (Link to Fabric)
Azure Blob Storage, OneDrive/SharePoint	Org account, SP	No	SharePoint shortcuts for files only
On-prem / network-restricted S3-compatible	Through on-premises data gateway	Yes	Gateway sizing matters

Syntax (REST):

```
POST https://api.fabric.microsoft.com/v1/workspaces/{wsId}/items/{lakehouseId}/shortcuts
{
  "path": "Tables/crm",
  "name": "accounts",
  "target": {
    "adlsGen2": {
      "location": "https://contosodatalake.dfs.core.windows.net",
      "subpath": "/crm/delta/accounts",
      "connectionId": "6a1d2c3e-aaaa-4bbb-9ccc-0d1e2f3a4b5c"
    }
  }
}
```

Example: create a OneLake shortcut so `lh_gold` can expose a warehouse dimension without copying it.

```
import sempy.fabric as fabric   # semantic-link, preinstalled in Fabric notebooks
client = fabric.FabricRestClient()
body = {
  "path": "Tables/shared", "name": "dim_store",
  "target": {"oneLake": {
      "workspaceId": "b2c1a9e0-1111-4c3e-9f00-2b6f0d1e7a10",
      "itemId": "9f3d7a22-2222-4a1b-8c55-71a0c9e4d201",   # wh_sales
      "path": "Tables/dbo/dim_store"}}
}
resp =
client.post(f"/v1/workspaces/{fabric.get_workspace_id()}/items/{fabric.get_lakehouse_id()}/shortcut
s", json=body)
print(resp.status_code)   # 201 Created
```

> **Trap:** deleting a shortcut deletes only the pointer. Deleting *files inside* a shortcut folder deletes them at the source if your identity has write access. Treat shortcut folders as read-only in pipelines.

Shortcut transformations and caching

Shortcut transformations (AI and file transformations) convert files reached through a shortcut, for example CSV or JSON to Delta, and keep the table in sync as files arrive. The shortcut cache, set at workspace level, stores remote S3/GCS reads for 1 to 28 days to cut egress cost.

```
Lakehouse > Files > New shortcut > Amazon S3 > choose folder
  > "Transform" > CSV to Delta table > target Tables/bronze/pos_receipts > Create
Workspace settings > OneLake > Enable cache for shortcuts > Retention: 7 days
```

Example: query the auto-converted table like any Delta table.

```sql
SELECT store_id, COUNT(*) AS receipts
FROM lh_bronze.bronze.pos_receipts
WHERE receipt_date = DATE '2026-10-26'
GROUP BY store_id;
```

Mirroring (overview)

How mirroring works

Mirroring replicates a database into OneLake as Delta tables in near real time, using the source's change feed (CDC, change tracking or replication logs). A mirrored database item comes with a SQL analytics endpoint and can feed Direct Lake directly. Storage for mirrored data is free up to a limit tied to the capacity size (1 TB per CU, so 64 TB for F64), and the replication compute is not charged to the capacity; queries against the mirror are.

Type	Sources (late 2026)	How it works
Database mirroring	Azure SQL DB, Azure SQL MI, SQL Server 2016-2025, Azure Cosmos DB, Azure Database for PostgreSQL, Snowflake, Oracle, SAP, Google BigQuery, SharePoint lists and more	Fabric reads the source change feed and writes Delta
Metadata mirroring	Azure Databricks Unity Catalog	Shortcuts to existing Delta tables; no data movement
Open mirroring	Any app or partner tool	You drop Parquet/CSV change files into a landing zone; Fabric merges them

```
Workspace > New item > Mirrored Azure SQL Database
  > connection (server, database, auth) > select tables (or "Mirror all data")
  > Connect > Mirror database   # initial snapshot, then continuous replication
```

Example: query the mirror through its SQL analytics endpoint and join it to warehouse data across databases.

```sql
SELECT c.customer_id, c.segment, SUM(f.net_amount) AS revenue_ytd
FROM   mirror_crm.dbo.customers AS c        -- mirrored Azure SQL DB
JOIN   wh_sales.dbo.fact_sales  AS f ON f.customer_id = c.customer_id
WHERE  f.order_date >= '2026-01-01'
GROUP BY c.customer_id, c.segment;
```

Mirroring is covered in more depth in the SQL database chapter.

OneLake catalog

Discover, govern and secure tabs

The OneLake catalog is the central place to find items across workspaces you can access. The **Explore** tab lists items with filters (type, owner, endorsement, domain, tags), the **Govern** tab shows insights and recommended actions for owners and admins (unlabeled items, missing descriptions, sharing), and the **Secure** tab gives a view of workspace roles and OneLake security roles. It replaced the older "OneLake data hub".

```
Fabric portal > OneLake (left rail) > Catalog
  Explore: filter Type = Lakehouse, Endorsement = Certified, Domain = Sales
  Govern:  "Your data estate" > items without sensitivity labels > Apply label
  Secure:  select lh_gold > Roles > review members of role "StoreManagers"
```

Example: the same discovery through the catalog search REST API (useful for inventory scripts; check the API reference for its current preview status).

```
POST https://api.fabric.microsoft.com/v1/catalog/search
{
  "search": "fact_sales",
  "filter": "Type eq 'Lakehouse' or Type eq 'Warehouse'"
}
```

Domains, tags and endorsement

Domains group workspaces by business area (Sales, Finance) so domain admins can delegate settings and the catalog can filter by them. **Tags** are admin-defined labels on items. **Endorsement** marks items as Promoted, Certified (by authorized reviewers) or Master data.

```
Admin portal > Domains > + Create domain "Sales" > assign workspaces ws-retail-*
Item settings > Endorsement > Certified   (needs certification rights set by admin)
```

Example: assign a workspace to a domain with the admin REST API.

```
POST https://api.fabric.microsoft.com/v1/admin/domains/{domainId}/assignWorkspaces
{ "workspacesIds": ["b2c1a9e0-1111-4c3e-9f00-2b6f0d1e7a10"] }
```

OneLake storage features

OneLake file explorer, APIs and storage tiering

The OneLake file explorer (Windows) syncs OneLake like OneDrive. Any ADLS Gen2 SDK can read and write OneLake. OneLake storage is billed separately from capacity (pay-as-you-go per GB), and soft-deleted data is retained for 7 days. In June 2026 Microsoft added automatic storage tiering based on access patterns; check current pricing for the cooler tiers before modelling costs.

```
from azure.storage.filedatalake import DataLakeServiceClient
from azure.identity import DefaultAzureCredential

svc = DataLakeServiceClient("https://onelake.dfs.fabric.microsoft.com",
credential=DefaultAzureCredential())
fs = svc.get_file_system_client("ws-retail-prod")            # workspace = file system
for p in fs.get_paths("lh_bronze.Lakehouse/Files/landing/stores"):
    print(p.name, p.content_length)
```

Example: upload a daily file from an on-prem script.

```
file = fs.get_file_client("lh_bronze.Lakehouse/Files/landing/stores/stores_2026-10-27.csv")
with open("stores_2026-10-27.csv", "rb") as fh:
    file.upload_data(fh, overwrite=True)
```

Table format interoperability (Delta and Iceberg)

OneLake can expose Delta tables as Apache Iceberg metadata and Iceberg tables as Delta (via shortcuts to Iceberg folders, with metadata virtualization), so Snowflake and other Iceberg engines can read Fabric data and Fabric can read Snowflake-managed Iceberg tables.

```
Lakehouse > Files or Tables > New shortcut > ADLS Gen2 / OneLake
  > point at an Iceberg table folder (metadata/ + data/)
  > Fabric generates a Delta log; table appears under Tables/
```

Example: Snowflake writes Iceberg tables into OneLake; Fabric reads them as Delta.

```
-- Snowflake side (external volume pointing at OneLake)
CREATE ICEBERG TABLE loyalty_events (member_id NUMBER, points NUMBER, event_ts TIMESTAMP_NTZ)
  CATALOG = 'SNOWFLAKE' EXTERNAL_VOLUME = 'onelake_vol' BASE_LOCATION = 'loyalty_events';
-- Fabric side, SQL analytics endpoint of lh_bronze after the shortcut exists
SELECT TOP 5 * FROM bronze.loyalty_events ORDER BY event_ts DESC;
```

Workspaces, capacities and licensing

Workspaces

Workspace design

A workspace is the unit of security, Git integration, deployment and capacity assignment. Experienced teams split workspaces by **layer and lifecycle** rather than putting everything in one place: separate storage workspaces (lakehouses, warehouse) from engineering workspaces (notebooks, pipelines) and reporting workspaces, each with dev/test/prod copies. That lets you grant analysts Viewer on gold without exposing bronze, and lets you move heavy Spark work to a different capacity.

Pattern	When to use	Gotcha
One workspace per stage (dev/test/prod)	Small team, one domain	Bronze and reports share a security boundary
Workspace per layer per stage (`ws-sales-bronze-prod`)	Enterprise, strict access	More shortcuts and connections to manage
Workspace per domain (data mesh)	Several business units own data	Need domains, endorsement and the catalog to stay discoverable

Syntax (REST):

```
POST https://api.fabric.microsoft.com/v1/workspaces
{ "displayName": "ws-retail-test", "capacityId": "0f9c2e5d-...-f64", "description": "Test stage" }
```

Example: create the three stage workspaces and assign a capacity in one script.

```
import requests
from azure.identity import DefaultAzureCredential
tok = DefaultAzureCredential().get_token("https://api.fabric.microsoft.com/.default").token
H = {"Authorization": f"Bearer {tok}"}
CAP = "0f9c2e5d-7a51-4d8e-b6a3-3c1f2e9d0f64"
for stage in ["dev", "test", "prod"]:
    r = requests.post("https://api.fabric.microsoft.com/v1/workspaces", headers=H,
                      json={"displayName": f"ws-retail-{stage}", "capacityId": CAP})
    print(stage, r.status_code, r.json().get("id"))
```

Workspace settings a data engineer owns

Setting	Where	Typical value
License mode / capacity	Workspace settings > License info	Fabric capacity `cap-retail-f64`
Spark pool, runtime, environment	Workspace settings > Data Engineering/Science > Spark settings	Starter pool, Runtime 2.0, default env `env_retail`
High concurrency for notebooks / pipelines	Spark settings > High concurrency	On
Workspace identity	Workspace settings > Workspace identity	Created; used for trusted workspace access
Git integration	Workspace settings > Git integration	Azure DevOps / GitHub branch
OneLake shortcut cache, OneLake diagnostics	Workspace settings > OneLake	On for S3/GCS shortcuts
Outbound access protection	Workspace settings > Network security	On for regulated workspaces

```
Workspace > Workspace settings > Data Engineering/Science > Spark settings
  Pool: Starter pool (Medium nodes)  Runtime: 2.0  Default environment: env_retail
  Jobs: "Reserve maximum cores for active Spark jobs" = Off   (optimistic admission)
```

Example: read the workspace Spark settings with REST before a migration.

```
curl -s -H "Authorization: Bearer $TOKEN" \
  "https://api.fabric.microsoft.com/v1/workspaces/$WS_ID/spark/settings" | jq '.pool, .environment'
```

Capacities and F-SKUs

Capacity units and SKU sizes

A capacity provides capacity units (CU). Every operation (Spark job, query, refresh, pipeline activity) consumes CU-seconds. F-SKUs are bought in Azure (pay-as-you-go, can pause and scale) or reserved (1- or 3-year commitment, cheaper, billed whether used or not). P-SKUs (Power BI Premium per capacity) are retired for new purchases; existing customers move to F-SKUs.

SKU	CU	Power BI equivalent	Spark VCores (1 CU = 2 VCores)	Notes
F2	2	none	4	Dev/test; free users can't view reports
F4 / F8 / F16 / F32	4-32	none	8-64	Below F64 report viewers need Pro/PPU licenses
F64	64	P1	128	Free (Fabric free) users can view content; Copilot historically from F64, now available on all paid F SKUs
F128 / F256 / F512	128-512	P2 / P3 / P4	256-1024	Large estates
F1024 / F2048	1024-2048	P5 / none	2048-4096	Largest
Trial	64	P1	128	60 days, per user

Spark bursting lets a single capacity use up to 3x its VCores for Spark (an F64 can run 384 VCores of Spark concurrently), and the extra use is paid back through smoothing.

Syntax (Azure CLI resource):

```
az resource create --resource-type "Microsoft.Fabric/capacities" \
  --name capretailf64 --resource-group rg-fabric --location westeurope \
  --properties '{"administration":{"members":["admin@contoso.com"]}}' \
  --api-version 2023-11-01 --is-full-object false \
  --set sku.name=F64 sku.tier=Fabric
```

Example: scale up for month-end, then back down (scripted in an Azure Automation runbook).

```
$cap = "/subscriptions/$sub/resourceGroups/rg-
fabric/providers/Microsoft.Fabric/capacities/capretailf64"
Invoke-AzRestMethod -Method PATCH -Path "$cap`?api-version=2023-11-01" `
  -Payload '{"sku":{"name":"F128","tier":"Fabric"}}'      # month-end close
# ... after close
Invoke-AzRestMethod -Method PATCH -Path "$cap`?api-version=2023-11-01" `
  -Payload '{"sku":{"name":"F64","tier":"Fabric"}}'
```

Trap: pausing a capacity bills any smoothed future usage and overage immediately, and makes all items in its workspaces unavailable (including OneLake reads through that capacity). Pause dev capacities at night; never pause the one serving reports.

Licensing for users

License	What it allows
Fabric (Free)	Create Fabric items in workspaces on an F capacity; view Power BI content only on F64+
Power BI Pro	Share and publish Power BI content; view content in capacities below F64
Premium Per User (PPU)	Premium Power BI features for named users, no Fabric items
Capacity (F-SKU)	Compute for all Fabric workloads

Example: a 40-person analytics team on F32 needs Pro licenses for everyone who views reports, because F32 is below F64. Cost check: if 300 viewers need Pro, moving to F64 is often cheaper than 300 Pro licenses. Run the numbers with your own price list.

```
Option A: F32 + 300 x Pro         Option B: F64 (free viewers) + 40 x Pro for authors
Compare annual cost of A and B; also compare CU headroom (F64 = 2x compute).
```

Smoothing and bursting

Fabric lets operations **burst** above the SKU's steady capacity and then **smooths** the cost over time. Interactive operations (report queries, SQL queries from users) are smoothed over a minimum of 5 minutes; background operations (Spark jobs, pipelines, refreshes, scheduled warehouse loads) are smoothed over 24 hours. This is why a 30-minute Spark job does not throttle anything by itself but a capacity that runs heavy jobs all day accumulates "carry-forward".

Worked example (F64 = 64 CU, so 64 × 30 = 1,920 CU-seconds per 30-second timepoint):

```
Nightly Spark job consumes 2,304,000 CU-seconds (background)
Smoothed over 24 h = 2,304,000 / 2,880 timepoints = 800 CU-s per timepoint
That is 800 / 1,920 = 41.7% of the F64 for the whole day.
Add interactive load averaging 50% → 91.7% utilization. Little headroom left.
```

Throttling stages

When smoothed usage exceeds 100% the excess becomes carry-forward debt measured in minutes of future capacity. Throttling is applied in stages:

Future usage already consumed	Stage	Effect
Up to 10 min	Overage protection	No throttling
10 to 60 min	Interactive delay	New interactive requests delayed 20 seconds
60 min to 24 h	Interactive rejection	Interactive requests rejected; background jobs still run
Over 24 h	Background rejection	All new requests rejected

Recovery is automatic as idle capacity burns down the debt. Options when throttled: wait, scale up (adds CU so debt burns faster), or pause/resume (pays the debt as a bill, then starts clean).

```
Capacity Metrics app > Compute page > "Throttling" tab
  Interactive delay / Interactive rejection / Background rejection charts (% of limit)
  > 100% on a line means that stage is active
```

Example: KQL over the workspace-monitoring or exported metrics to find which items caused the debt (pattern; table names depend on your export).

```kql
// Top CU consumers over the last 24h from an exported capacity-usage table
CapacityUsage
| where Timestamp > ago(24h)
| summarize CU_s = sum(CUSeconds) by WorkspaceName, ItemName, OperationName
| top 10 by CU_s desc
```

Surge protection and capacity guardrails

Surge protection lets admins set limits so background jobs are rejected when the 24-hour background usage passes a threshold, protecting interactive users. Newer controls also let admins cap a single workspace's share. Warehouse has its own guardrails against unbounded bursting on small SKUs.

```
Admin portal > Capacity settings > cap-retail-f64 > Surge protection
  Background rejection threshold: 80%   Recovery threshold: 60%
```

Example: you set 80%. At 2 p.m. a data scientist launches a big backfill; once 24-hour background usage passes 80%, new background jobs are rejected with a capacity error while Power BI users keep working. The nightly pipeline must have retries with backoff to survive this.

```
{ "name": "Copy orders", "type": "Copy",
  "policy": { "timeout": "0.02:00:00", "retry": 3, "retryIntervalInSeconds": 600 } }
```

Capacity Metrics app

The Microsoft Fabric Capacity Metrics app is the official tool for CU usage. Install it from AppSource, point it at a capacity ID, and use the Compute page (utilization by item and operation, throttling, overages) and the Storage page. It shows 14 days of history.

```
Install: Apps > Get apps > "Microsoft Fabric Capacity Metrics" > connect with Capacity ID, UTC
offset
Compute page: ribbon chart by workload > select a timepoint > "Explore" > timepoint detail
  Interactive operations table / Background operations table (CU (s), duration, user, item)
```

Example: diagnosing a 2 p.m. slowdown. Open the timepoint, sort background operations by "Timepoint CU (s)", find `nb_backfill_2025` consuming 38% of the timepoint, and move it to the dev capacity or to off-hours.

```
Timepoint 2026-10-26 14:00:30  Utilization 112%
 Background: nb_backfill_2025 (Notebook)  38.2%   user dana@contoso.com
             sm_sales refresh             21.0%
 Interactive: rpt_sales DAX queries       44.6%
```

Autoscale billing for Spark

Autoscale billing for Spark moves Spark jobs off the shared capacity to pay-as-you-go serverless billing, up to a CU limit you set. Spark then no longer consumes or throttles the capacity, which protects reports and SQL.

```
Azure portal / Fabric admin > Capacity settings > Data Engineering/Science
  Autoscale billing for Spark: On   Maximum CU: 256
```

Example decision: an F64 where Spark causes 70% of usage. Enabling autoscale billing could let you drop to F32 for SQL and BI while Spark pays per use; compare a month of CU-hours at the PAYG rate before deciding.

```
Spark CU-hours last 30 days (from Metrics app): 18,400
Cost if billed per use = 18,400 x <PAYG rate per CU-hour>   vs   F64 - F32 monthly difference
```

Lakehouse

Lakehouse structure

Tables vs Files

A lakehouse has two top-level areas. **Tables** holds managed Delta tables that are registered in the metastore and visible to the SQL analytics endpoint and Direct Lake. **Files** holds anything else: raw CSV, JSON, images, Parquet that is not yet a table. Files does not show up in SQL. A common bronze pattern is to land raw files in `Files/landing/` and then load them into `Tables/bronze`.

Area	Visible to SQL endpoint	Format	Use
`Tables/`	Yes	Delta only	Curated, queryable tables
`Files/`	No (read via Spark or OPENROWSET)	Any	Landing zone, archives, ML assets
Shortcut in `Tables/`	Yes, if target is Delta	Delta	Zero-copy tables
Shortcut in `Files/`	No	Any	External raw data

Syntax:

```
spark.read.format("csv").option("header", "true").load("Files/landing/orders/*.csv")
df.write.format("delta").saveAsTable("bronze.orders_raw")       # managed table
df.write.format("delta").save("Files/archive/orders_2026_10")   # just files, not a table
```

Example: land a day of CSV files and register them as a bronze table.

```
from pyspark.sql import functions as F

raw = (spark.read.option("header", "true").option("inferSchema", "false")
       .csv("Files/landing/orders/2026/10/26/*.csv")
       .withColumn("_ingest_ts", F.current_timestamp())
       .withColumn("_source_file", F.col("_metadata.file_path")))
raw.write.mode("append").format("delta").saveAsTable("lh_bronze.bronze.orders_raw")
```

Managed vs external (unmanaged) tables

A managed table's data lives under the lakehouse `Tables/` folder and is deleted with the table. An external table only registers metadata over a location you control (for example in `Files/` or ADLS), and dropping it leaves the files. Fabric prefers managed tables; external tables do not appear in the SQL analytics endpoint unless placed under `Tables/` via shortcut.

```
-- Managed
CREATE TABLE silver.customers (customer_id BIGINT, name STRING, segment STRING) USING DELTA;
-- External
CREATE TABLE bronze.weblogs_ext USING DELTA LOCATION 'Files/external/weblogs';
```

Example: check what kind of table you have before dropping it.

```
DESCRIBE EXTENDED silver.customers;   -- "Type: MANAGED" or "EXTERNAL", plus Location
DROP TABLE IF EXISTS bronze.weblogs_ext;  -- files in Files/external/weblogs remain
```

Lakehouse schemas

Schema-enabled lakehouses (the default for new lakehouses) organise tables into schemas such as `dbo`, `sales`, `finance`, giving four-part names in Spark (`workspace.lakehouse.schema.table`) and three-part names in SQL. Schemas also let you put a shortcut to a whole folder of tables under one schema.

```
CREATE SCHEMA IF NOT EXISTS sales;
CREATE TABLE sales.orders (order_id BIGINT, order_date DATE, customer_id BIGINT, order_total
DECIMAL(18,2));
SELECT * FROM `ws-retail-prod`.lh_silver.sales.orders LIMIT 10;   -- cross-workspace, Spark
```

Example: organise silver by subject area and query from another lakehouse's notebook.

```
for s in ["sales", "customer", "inventory"]:
    spark.sql(f"CREATE SCHEMA IF NOT EXISTS lh_silver.{s}")
df = spark.sql("""
  SELECT o.order_id, c.segment
  FROM lh_silver.sales.orders o
  JOIN lh_silver.customer.customers c USING (customer_id)
""")
```

SQL analytics endpoint

Every lakehouse gets a read-only T-SQL endpoint, generated automatically. It reads the Delta tables under `Tables/`. You can create views, inline table-valued functions, stored procedures and grant permissions (GRANT/DENY, RLS, CLS, DDM) on it, but you cannot INSERT/UPDATE/DELETE tables. Metadata sync from Delta to the endpoint is automatic and usually takes seconds; you can force it with the refresh API when a pipeline needs to query right after a write.

```
CREATE VIEW sales.v_daily_revenue AS
SELECT order_date, SUM(order_total) AS revenue, COUNT(*) AS orders
FROM sales.orders GROUP BY order_date;
```

Example: force a metadata refresh after a notebook writes, before a downstream SQL step reads.

```
POST
https://api.fabric.microsoft.com/v1/workspaces/{wsId}/sqlEndpoints/{sqlEndpointId}/refreshMetadata
Content-Type: application/json
{ "timeout": { "timeUnit": "Minutes", "value": 2 } }
```

Trap: the SQL analytics endpoint is read-only for tables. "Write with T-SQL" questions point to the Warehouse. Also, tables with column names containing spaces or unsupported types may not appear; use column mapping or rename columns.

Loading data into a lakehouse

Method	Code	Best for
Notebook (PySpark/Spark SQL)	Yes	Complex transforms, large volume
Pipeline Copy activity / Copy job	Low	Bulk or incremental copies from 100+ connectors
Dataflow Gen2	Low (Power Query)	Business-user transforms, small/medium data
"Load to Tables" (right-click file)	No	One-off CSV/Parquet to Delta
Shortcut / mirroring	No	No copy needed
Eventstream	No	Streaming events to Delta

```
Lakehouse explorer > Files/landing/stores.csv > ... > Load to Tables > New table "stores"
```

Example: the same load in code with explicit schema (faster and safer than inferSchema).

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DateType
schema = StructType([
    StructField("store_id", IntegerType(), False),
    StructField("store_name", StringType()),
    StructField("region", StringType()),
    StructField("opened_on", DateType())])
(spark.read.schema(schema).option("header", True).csv("Files/landing/stores.csv")
      .write.mode("overwrite").saveAsTable("bronze.stores"))
```

Medallion architecture

Bronze, silver, gold

The medallion architecture splits data by quality. **Bronze** holds raw data as received (append-only, with ingest metadata). **Silver** holds cleaned, typed, deduplicated and conformed data. **Gold** holds business-ready models: star schemas and aggregates. In Fabric each layer is often its own lakehouse (and sometimes workspace), with the gold layer either a lakehouse or a warehouse.

Layer	Content	Write pattern	Typical item
Bronze	Raw, schema-on-read, full history	Append, files + Delta	Lakehouse
Silver	Typed, deduped, validated, SCD1	MERGE upsert	Lakehouse
Gold	Facts, dimensions (SCD2), aggregates	MERGE / overwrite partitions	Lakehouse or Warehouse

```
Sources ──Copy job / pipeline / mirroring / Eventstream──▶ lh_bronze
lh_bronze ──notebooks (PySpark, MERGE)──▶ lh_silver
lh_silver ──notebooks or T-SQL procs──▶ lh_gold / wh_sales ──Direct Lake──▶ sm_sales
```

Example: a silver notebook that dedupes and types bronze orders.

```
from pyspark.sql import functions as F, Window

b = spark.read.table("lh_bronze.bronze.orders_raw").where("_ingest_ts >= current_date() - 1")
w = Window.partitionBy("order_id").orderBy(F.col("_ingest_ts").desc())
clean = (b.withColumn("rn", F.row_number().over(w)).where("rn = 1").drop("rn")
          .select(F.col("order_id").cast("bigint"),
                  F.to_date("order_date", "yyyy-MM-dd").alias("order_date"),
                  F.col("customer_id").cast("bigint"),
                  F.col("order_total").cast("decimal(18,2)"),
                  F.upper(F.trim("order_status")).alias("order_status"),
                  "_ingest_ts"))
clean.createOrReplaceTempView("src_orders")
spark.sql("""
MERGE INTO lh_silver.sales.orders t
USING src_orders s ON t.order_id = s.order_id
WHEN MATCHED AND s._ingest_ts > t._ingest_ts THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
""")
```

Materialized lake views

Materialized lake views (MLVs) let you declare a silver or gold table as a Spark SQL query. Fabric stores the result as Delta, works out the dependency graph between views, refreshes them in order (incrementally where possible) and enforces data-quality constraints. Spark SQL MLVs are GA since March 2026; PySpark authoring is in preview.

Syntax:

```
CREATE MATERIALIZED LAKE VIEW [IF NOT EXISTS] schema.view_name
[( CONSTRAINT name CHECK (condition) ON MISMATCH { DROP | FAIL } , ... )]
[PARTITIONED BY (col)]
[COMMENT 'text']
[TBLPROPERTIES ("key"="value")]
AS select_statement;

REFRESH MATERIALIZED LAKE VIEW schema.view_name [FULL];
```

Example: a silver customer view with quality rules and a gold aggregate that depends on it.

```
CREATE MATERIALIZED LAKE VIEW IF NOT EXISTS silver.customers_clean
( CONSTRAINT valid_email CHECK (email LIKE '%@%') ON MISMATCH DROP,
  CONSTRAINT has_id CHECK (customer_id IS NOT NULL) ON MISMATCH FAIL )
COMMENT 'Deduped CRM customers'
AS SELECT customer_id, lower(trim(email)) AS email, initcap(name) AS name, segment
   FROM bronze.crm_customers
   QUALIFY row_number() OVER (PARTITION BY customer_id ORDER BY updated_at DESC) = 1;

CREATE MATERIALIZED LAKE VIEW IF NOT EXISTS gold.revenue_by_segment
AS SELECT c.segment, o.order_date, SUM(o.order_total) AS revenue
   FROM silver.orders o JOIN silver.customers_clean c USING (customer_id)
   GROUP BY c.segment, o.order_date;
```

Schedule refresh from Lakehouse > Manage materialized lake views (lineage view shows the DAG and run history).

> **Best practice:** use MLVs for SQL-expressible transformations with clear dependencies. Keep complex procedural logic (API calls, ML scoring, SCD2 with custom rules) in notebooks.

Lakehouse utilities

Table maintenance from the UI and API

Lakehouse explorer offers "Maintenance" on a table, which runs OPTIMIZE (with optional V-Order) and VACUUM with a retention threshold. The same is available as a background job through REST, which is handy for scheduling maintenance across many tables.

```
POST https://api.fabric.microsoft.com/v1/workspaces/{wsId}/lakehouses/{lhId}/jobs/instances?
jobType=TableMaintenance
{
  "executionData": {
    "tableName": "orders", "schemaName": "sales",
    "optimizeSettings": { "vOrder": "true", "zOrderBy": ["customer_id"] },
    "vacuumSettings": { "retentionPeriod": "7.01:00:00" }
  }
}
```

Example: loop over all silver tables in a notebook instead.

```
tables = [r.tableName for r in spark.sql("SHOW TABLES IN lh_silver.sales").collect()]
for t in tables:
    spark.sql(f"OPTIMIZE lh_silver.sales.{t}")
    spark.sql(f"VACUUM lh_silver.sales.{t} RETAIN 168 HOURS")
```

Default lakehouse and mounts

A notebook's default (pinned) lakehouse defines what relative paths like `Files/...` and unqualified table names resolve to. You can attach several lakehouses but only one is default. In pipelines and deployment you can change it with `%%configure` or the notebook's parameters.

```
%%configure -f
{ "defaultLakehouse": { "name": "lh_silver", "id": "7d1c...", "workspaceId": "b2c1..." } }
```

Example: mount another lakehouse to use local file APIs (pandas, open()).

```
from notebookutils import mssparkutils  # also available as notebookutils.fs
notebookutils.fs.mount("abfss://ws-retail-
prod@onelake.dfs.fabric.microsoft.com/lh_bronze.Lakehouse",
                       "/lh_bronze")
local = notebookutils.fs.getMountPath("/lh_bronze")
import pandas as pd
pdf = pd.read_csv(f"{local}/Files/landing/stores.csv")
```

Spark and notebooks

Spark compute

Fabric runtimes

A runtime bundles Spark, Delta, Python, Java/Scala and default libraries. You pick it per workspace or per environment.

Runtime	Spark	Delta	Python	Java / Scala	Status (Oct 2026)
1.2	3.4	2.4	3.10	11 / 2.12	End of support path; migrate
1.3	3.5	3.2	3.11	11 / 2.12	GA, widely used
2.0	4.1	4.2	3.13	21 / 2.13	GA; default for new workspaces from late Sept 2026

```
Workspace settings > Data Engineering/Science > Spark settings > Environment > Runtime version: 2.0
or: Environment item > Spark compute > Runtime: 2.0  (overrides workspace default)
```

Example: confirm runtime versions in a session before deploying a wheel built for Python 3.11.

```python
import sys, pyspark, delta
print(sys.version)              # 3.13.x on Runtime 2.0
print(pyspark.__version__)      # 4.1.x
print(spark.conf.get("spark.fabric.runtime.version", "n/a"))
```

Trap: moving to Runtime 2.0 changes Python to 3.13 and Scala to 2.13. Wheels and JARs compiled for older versions fail at import. Rebuild custom libraries and retest environments before switching the workspace default.

Starter pools and custom pools

Starter pools are pre-warmed Medium-node clusters that Microsoft keeps running, so sessions start in seconds. **Custom pools** let you choose node size (Small 4 VCores to XX-Large 64 VCores), autoscale min/max nodes and dynamic executor allocation; they start in a few minutes unless you use **custom live pools** (GA September 2026), which keep custom pools pre-warmed. You pay only while sessions run.

Pool	Start time	Configurable	Gotcha
Starter pool	~5-10 s	Max nodes, executors	Medium nodes only; custom libraries add start time
Custom pool	~2-5 min	Node size, autoscale, executors	Cold start; library install adds more
Custom live pool	Fast (pre-warmed)	As custom pool	Check how idle warm capacity is billed

```http
POST https://api.fabric.microsoft.com/v1/workspaces/{wsId}/spark/pools
{
  "name": "pool_large_etl", "nodeFamily": "MemoryOptimized", "nodeSize": "Large",
  "autoScale": { "enabled": true, "minNodeCount": 1, "maxNodeCount": 10 },
  "dynamicExecutorAllocation": { "enabled": true, "minExecutors": 1, "maxExecutors": 9 }
}
```

Example: size a pool for a 2 TB nightly join. Large nodes have 16 VCores and 128 GB; 10 nodes give 160 VCores, which is 80 CU of Spark while the job runs. On an F64 (128 base VCores, 384 with bursting) that fits, but it competes with other jobs.

```
Need: shuffle ~2 TB → aim for ~200 MB partitions → ~10,000 shuffle partitions over time
Pool: Large x 10 nodes = 160 VCores; spark.sql.shuffle.partitions = 640 (4x cores), AQE on
```

Environments

An environment item packages runtime version, Spark properties, compute settings and libraries (public PyPI/Conda packages, custom wheels/JARs). Attach it to notebooks and Spark job definitions, or set it as the workspace default. Publishing an environment builds it once so sessions start faster.

```yaml
# environment.yml uploaded to the environment item (Public libraries > Upload)
dependencies:
  - pip:
      - great-expectations==1.3.0
      - azure-storage-blob==12.24.0
```

Example: environment properties for an ETL team.

```
env_retail_etl
  Runtime: 2.0
  Spark properties:
    spark.sql.shuffle.partitions = auto      (AQE decides)
    spark.native.enabled = true              (native execution engine)
    spark.databricks.delta.optimizeWrite.enabled = true
  Custom libraries: contoso_dq-1.4.0-py3-none-any.whl
  Compute: pool_large_etl, driver 8 cores / 56 GB
```

Session configuration with %%configure

`%%configure` sets session-level Spark settings at the start of a notebook (driver/executor sizes, default lakehouse, environment, conf). When a pipeline runs the notebook, values can be parameterized.

```
%%configure -f
{
  "driverMemory": "28g", "driverCores": 4,
  "executorMemory": "28g", "executorCores": 4, "numExecutors": 6,
  "conf": { "spark.sql.shuffle.partitions": "200" },
  "defaultLakehouse": { "name": { "parameterName": "lakehouseName", "defaultValue": "lh_silver" } }
}
```

Example: a pipeline Notebook activity passes `lakehouseName = lh_silver_test` in the test workspace, so the same notebook targets the right lakehouse per stage.

```
{ "type": "TridentNotebook", "name": "Run silver orders",
  "typeProperties": { "notebookId": "...", "workspaceId": "...",
    "parameters": { "lakehouseName": { "value": "lh_silver_test", "type": "string" } } } }
```

High concurrency mode

High concurrency mode lets several notebooks share one Spark session (each gets its own REPL), so the second notebook starts instantly and the capacity is billed once. It works for interactive notebooks and for notebooks run from pipelines (enable both in workspace settings). Sessions are shared only between notebooks by the same user with the same default lakehouse, environment and Spark config.

```
Workspace settings > Spark settings > High concurrency
  [x] For notebooks      [x] For pipeline running multiple notebooks
Pipeline Notebook activity > Settings > Session tag: "silver_batch"
```

Example: a pipeline runs eight silver notebooks in a ForEach with the same session tag. Without high concurrency each starts its own session (eight starts, eight drivers); with it, one session serves all eight.

```
{ "type": "TridentNotebook", "typeProperties": { "notebookId": "@item().notebookId",
  "sessionTag": "silver_batch" } }
```

Native execution engine

The native execution engine runs Spark SQL and DataFrame operators in vectorized C++ (based on Apache Gluten and Velox) instead of the JVM. It needs no code changes and falls back to the JVM for unsupported operators. It is GA on Runtime 1.3 and 2.0, and benchmarks from Microsoft show large speed-ups on Parquet/Delta-heavy queries. It does not accelerate Python UDFs.

```
%%configure
{ "conf": { "spark.native.enabled": "true" } }
```

```
SET spark.native.enabled = true;   -- per session in Spark SQL
```

Example: check whether a query actually ran natively. Look for operators prefixed with `Velox` / `Native` / `Transformer` in the physical plan, and fallbacks in the Spark UI "Gluten" tab.

```
df = spark.sql("SELECT store_id, SUM(net_amount) s FROM lh_gold.gold.fact_sales GROUP BY store_id")
df.explain()   # "...HashAggregateTransformer... FileSourceScanExecTransformer..." = native
```

Autotune and other Spark settings

Autotune uses past runs to tune `spark.sql.shuffle.partitions`, `spark.sql.autoBroadcastJoinThreshold` and `spark.sql.files.maxPartitionBytes` per query. Optimize write (`spark.databricks.delta.optimizeWrite.enabled`) reduces small files by shuffling before the write.

```
spark.conf.set("spark.ms.autotune.enabled", "true")
spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true")
spark.conf.set("spark.databricks.delta.optimizeWrite.binSize", "1g")
```

Example: a streaming-to-silver job that wrote 4 MB files is fixed by turning on optimize write and scheduling OPTIMIZE nightly.

```
spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true")
events.write.mode("append").saveAsTable("lh_silver.web.clicks")
```

Notebooks

Notebook features

Fabric notebooks support PySpark, Spark SQL (`%%sql`), Scala (`%%spark`), SparkR (`%%sparkr`) and pure Python notebooks (single-node, no Spark, cheaper for small jobs). Features include parameter cells, `%run` to include another notebook, Copilot, a variable explorer, a resources folder, co-editing, scheduling and Git support (stored as `.py` with metadata).

Magic	Purpose
`%%sql`	Run Spark SQL cell
`%%configure`	Session config
`%run nb_common_functions`	Inline another notebook's code
`%pip install pkg`	Session-scoped library install
`%%tsql` (Python notebooks)	Run T-SQL against a warehouse/endpoint

```
# Parameters cell (toggle "parameter cell" in the cell menu)
run_date = "2026-10-26"
source_system = "pos"
```

Example: a reusable silver notebook that reads its parameters and exits with a value for the pipeline.

```
%run nb_common_functions        # defines log_run(), get_watermark()

rows = (spark.read.table("lh_bronze.bronze.orders_raw")
        .where(f"order_date = '{run_date}'").count())
log_run("silver_orders", run_date, rows)
notebookutils.notebook.exit(str(rows))   # pipeline reads @activity('nb').output.result.exitValue
```

Python notebooks (single node)

Python notebooks run on a 2-VCore container by default without Spark, and include pandas, DuckDB, Polars and the `deltalake` library. For small files (megabytes to a few GB) they are cheaper and faster to start than Spark.

```
import duckdb, polars as pl
from deltalake import write_deltalake
```

Example: transform a 300 MB reference file with Polars and write a Delta table.

```
import polars as pl
from deltalake import write_deltalake
df = pl.read_csv("/lakehouse/default/Files/landing/products.csv")
df = df.with_columns(pl.col("list_price").cast(pl.Decimal(18, 2)))
write_deltalake("/lakehouse/default/Tables/dbo/products", df.to_arrow(), mode="overwrite")
```

notebookutils (formerly mssparkutils)

`notebookutils` is the built-in utility library: file system (`fs`), notebook orchestration (`notebook.run`, `runMultiple`), credentials (`credentials.getSecret` from Key Vault, `getToken`), lakehouse management and runtime context.

```
notebookutils.fs.ls("Files/landing")
notebookutils.fs.cp("Files/landing/a.csv", "Files/archive/a.csv")
secret = notebookutils.credentials.getSecret("https://kv-retail.vault.azure.net/", "crm-api-key")
notebookutils.notebook.run("nb_silver_customers", 1800, {"run_date": "2026-10-26"})
```

Example: run a DAG of notebooks in parallel with dependencies.

```
dag = {
  "activities": [
    {"name": "customers", "path": "nb_silver_customers", "timeoutPerCellInSeconds": 600,
     "args": {"run_date": run_date}},
    {"name": "products",  "path": "nb_silver_products", "args": {"run_date": run_date}},
    {"name": "orders",    "path": "nb_silver_orders", "args": {"run_date": run_date},
     "dependencies": ["customers", "products"]},
  ],
  "concurrency": 3, "timeoutInSeconds": 3600
}
results = notebookutils.notebook.runMultiple(dag)
```

PySpark essentials

Reading and writing

```
spark.read.format("csv|json|parquet|delta").option(k, v).schema(s).load(path)
spark.read.table("lakehouse.schema.table")
df.write.mode("append|overwrite|error|ignore").format("delta") \
  .partitionBy(col).option("mergeSchema", "true").saveAsTable(name)
```

Example: read nested JSON from an API landing folder and flatten it.

```
from pyspark.sql import functions as F
j = spark.read.option("multiLine", True).json("Files/landing/crm/accounts/2026-10-26/*.json")
flat = (j.select(F.explode("value").alias("a"))
         .select("a.id", "a.name", F.col("a.address.city").alias("city"),
                 F.col("a.address.country").alias("country"),
                 F.to_timestamp("a.modifiedOn").alias("modified_on")))
flat.write.mode("append").saveAsTable("lh_bronze.bronze.crm_accounts")
```

Transformations

```
df.select(...).withColumn(name, expr).filter(cond).groupBy(...).agg(...)
df.join(other, on=keys, how="inner|left|anti|semi").dropDuplicates([keys])
F.when(cond, v).otherwise(v2); F.coalesce(a, b); F.date_trunc("month", c)
```

Example: build a daily store-level fact with a broadcast join to a small dimension.

```
from pyspark.sql import functions as F
orders = spark.table("lh_silver.sales.order_lines")
stores = spark.table("lh_silver.inventory.stores")          # 1,200 rows
fact = (orders.join(F.broadcast(stores), "store_id", "left")
        .groupBy("order_date", "store_id", "region")
        .agg(F.sum("net_amount").alias("revenue"),
             F.countDistinct("order_id").alias("orders"),
             F.sum(F.when(F.col("is_return"), F.col("net_amount")).otherwise(0)).alias("returns")))
fact.write.mode("overwrite").option("replaceWhere", "order_date >= '2026-10-01'") \
    .saveAsTable("lh_gold.gold.fact_store_daily")
```

Window functions

```
w = Window.partitionBy("customer_id").orderBy("order_date").rowsBetween(Window.unboundedPreceding,
0)
F.row_number().over(w); F.lag("x", 1).over(w); F.sum("x").over(w)
```

Example: running revenue and days since previous order per customer.

```
from pyspark.sql import Window, functions as F
w = Window.partitionBy("customer_id").orderBy("order_date")
cust = (spark.table("lh_silver.sales.orders")
  .withColumn("running_rev", F.sum("order_total").over(w.rowsBetween(Window.unboundedPreceding,
0)))
  .withColumn("days_since_prev", F.datediff("order_date", F.lag("order_date").over(w))))
```

Spark SQL

Spark SQL works on the same tables. Use `%%sql` cells, `spark.sql()`, or temp views to mix languages.

```
CREATE OR REPLACE TEMP VIEW v AS SELECT ...;
INSERT INTO t SELECT ...;   INSERT OVERWRITE t PARTITION (p='x') SELECT ...;
```

Example: top 3 products per category with QUALIFY.

```
%%sql
SELECT category, product_name, revenue
FROM (
  SELECT p.category, p.product_name, SUM(l.net_amount) AS revenue
  FROM lh_silver.sales.order_lines l JOIN lh_silver.inventory.products p USING (product_id)
  WHERE l.order_date >= '2026-10-01'
  GROUP BY p.category, p.product_name)
QUALIFY ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) <= 3;
```

Structured Streaming

Spark Structured Streaming in Fabric reads from Event Hubs, Kafka, Eventstream (custom endpoint) or Delta tables and writes to Delta with checkpoints. Run long streams from a Spark job definition with a retry policy rather than from an interactive notebook.

```
(spark.readStream.format(fmt).options(**opts).load()
   .writeStream.format("delta").option("checkpointLocation", path)
   .trigger(processingTime="1 minute" | availableNow=True).toTable(name))
```

Example: incremental bronze-to-silver using a Delta stream with `availableNow` (batch-like, exactly once).

```
q = (spark.readStream.table("lh_bronze.bronze.clicks_raw")
       .selectExpr("CAST(event_id AS STRING) AS event_id", "user_id", "page",
                   "to_timestamp(event_ts) AS event_ts")
       .withWatermark("event_ts", "30 minutes")
       .dropDuplicates(["event_id", "event_ts"]))          # dedupe late replays
(q.writeStream.format("delta")
   .option("checkpointLocation", "Files/checkpoints/silver_clicks")
   .trigger(availableNow=True)
   .toTable("lh_silver.web.clicks"))
```

Spark job definitions and Livy API

A Spark job definition (SJD) runs a main `.py` / `.jar` file with arguments, reference files and a retry policy, which suits streaming and packaged batch code. The Livy API lets external tools submit Spark batches or sessions to Fabric over REST (High concurrency for Livy went GA in September 2026).

```
POST https://api.fabric.microsoft.com/v1/workspaces/{wsId}/lakehouses/{lhId}/livyapi/versions/2023-12-01/batches
{ "name": "orders_backfill", "file": "abfss://.../Files/jobs/backfill.py",
  "args": ["--from", "2025-01-01", "--to", "2025-12-31"],
  "conf": { "spark.native.enabled": "true" } }
```

Example: SJD settings for the clickstream job.

```
sjd_clicks_stream
  Main definition file: Files/jobs/clicks_stream.py   Language: PySpark
  Command line args: --checkpoint Files/checkpoints/clicks --trigger 30s
  Lakehouse reference: lh_silver (default)
  Optimization > Retry policy: on, max retries 10, 60 s between retries
```

Delta Lake operations

Writing and changing data

MERGE (upsert)

MERGE applies inserts, updates and deletes from a source to a target in one atomic commit. It is the workhorse of silver and gold loads. Performance depends on how many target files the join must touch, so filter the target with partition or clustering columns in the ON clause.

Syntax:

```
MERGE INTO target t
USING source s
ON t.key = s.key [AND t.partition_col = s.partition_col]
WHEN MATCHED [AND cond] THEN UPDATE SET col = s.col | UPDATE SET *
WHEN MATCHED [AND cond] THEN DELETE
WHEN NOT MATCHED [AND cond] THEN INSERT (cols) VALUES (...) | INSERT *
WHEN NOT MATCHED BY SOURCE [AND cond] THEN UPDATE SET ... | DELETE;
```

Example: upsert CRM customers and soft-delete those missing from a full extract.

```
MERGE INTO lh_silver.customer.customers AS t
USING stg_crm_customers AS s
ON t.customer_id = s.customer_id
WHEN MATCHED AND (t.email <> s.email OR t.segment <> s.segment OR t.is_deleted) THEN
  UPDATE SET t.email = s.email, t.segment = s.segment, t.is_deleted = false,
             t.updated_at = current_timestamp()
WHEN NOT MATCHED THEN
  INSERT (customer_id, email, segment, is_deleted, updated_at)
  VALUES (s.customer_id, s.email, s.segment, false, current_timestamp())
WHEN NOT MATCHED BY SOURCE AND t.is_deleted = false THEN
  UPDATE SET t.is_deleted = true, t.updated_at = current_timestamp();
```

The same with the Python API:

```
from delta.tables import DeltaTable
tgt = DeltaTable.forName(spark, "lh_silver.customer.customers")
(tgt.alias("t").merge(stg.alias("s"), "t.customer_id = s.customer_id")
    .whenMatchedUpdate(condition="t.email <> s.email OR t.segment <> s.segment",
                       set={"email": "s.email", "segment": "s.segment", "updated_at":
"current_timestamp()"})
    .whenNotMatchedInsertAll()
    .execute())
```

> **Trap:** MERGE fails with "multiple source rows matched" if the source has duplicate keys. Deduplicate the source first (row_number over the key, keep the latest).

UPDATE, DELETE, INSERT OVERWRITE, replaceWhere

```
UPDATE t SET col = expr WHERE cond;
DELETE FROM t WHERE cond;
INSERT OVERWRITE t SELECT ...;                      -- replace whole table
```

```
df.write.mode("overwrite").option("replaceWhere", "order_date = '2026-10-26'").saveAsTable(t)
```

Example: GDPR erasure followed by a partition reload.

```
DELETE FROM lh_silver.customer.customers WHERE customer_id IN (SELECT customer_id FROM
ops.gdpr_requests WHERE status = 'APPROVED');
-- deletion vectors make this fast; VACUUM later removes the old files physically
```

```
day = spark.table("lh_silver.sales.orders").where("order_date = '2026-10-26'")
(build_fact(day).write.mode("overwrite")
   .option("replaceWhere", "order_date = '2026-10-26'")
   .saveAsTable("lh_gold.gold.fact_sales"))
```

Table layout and maintenance

OPTIMIZE and bin-compaction

Frequent small writes create many small files, which slow reads. OPTIMIZE rewrites them into larger files (target around 128 MB to 1 GB). It is idempotent and can be limited to recent partitions.

```
OPTIMIZE table_name [WHERE partition_predicate] [ZORDER BY (col1, col2)] [VORDER];
```

Example: compact yesterday's partition of a large fact table each morning.

```
OPTIMIZE lh_gold.gold.fact_sales WHERE order_date >= date_sub(current_date(), 1) VORDER;
```

Z-ORDER

Z-ordering co-locates rows with similar values of the chosen columns in the same files, so file-level min/max statistics let the engine skip files. Pick high-cardinality columns that appear in filters and joins (customer_id, device_id). It only helps when combined with OPTIMIZE and is not incremental.

```
OPTIMIZE lh_silver.sales.orders ZORDER BY (customer_id);
```

Example: measure the effect.

```
spark.sql("OPTIMIZE lh_silver.sales.orders ZORDER BY (customer_id)")
df = spark.sql("SELECT * FROM lh_silver.sales.orders WHERE customer_id = 1048213")
df.collect()
# Spark UI > SQL tab > scan node: "number of files read" drops (e.g. 1,840 -> 12)
```

Liquid clustering

Delta liquid clustering replaces partitioning and Z-order with clustering keys that can change over time and are applied incrementally by OPTIMIZE. It is supported in recent Fabric runtimes with Delta 3.x/4.x; confirm the reader support of every engine that reads the table (SQL endpoint, Direct Lake) before using it on serving tables.

```
CREATE TABLE gold.fact_events (event_id BIGINT, device_id STRING, event_date DATE, value DOUBLE)
USING DELTA CLUSTER BY (device_id, event_date);
ALTER TABLE gold.fact_events CLUSTER BY (event_date);
OPTIMIZE gold.fact_events;
```

Example: a telemetry table queried by device and date, previously over-partitioned by device (100k tiny partitions), rebuilt with clustering.

```
CREATE TABLE lh_silver.iot.readings_v2 USING DELTA CLUSTER BY (device_id, reading_date)
AS SELECT * FROM lh_silver.iot.readings;
```

Partitioning

Partitioning splits a table into folders by column value. In Fabric it pays off only for large tables (hundreds of GB and up) with a low-cardinality filter column, typically a date. Each partition should hold at least 1 GB. Over-partitioning creates small files and slows Direct Lake.

```
df.write.partitionBy("order_year", "order_month").format("delta").saveAsTable(t)
```

Example: partition a 3 TB fact by month, not by day.

```
from pyspark.sql import functions as F
(spark.table("lh_silver.sales.order_lines")
   .withColumn("order_month", F.date_format("order_date", "yyyy-MM"))
   .write.partitionBy("order_month").mode("overwrite")
   .saveAsTable("lh_gold.gold.fact_sales"))
```

VACUUM

VACUUM physically deletes files no longer referenced by the current table version and older than the retention period (default 7 days). It frees storage but removes the ability to time travel beyond the retention period. Fabric blocks retention under 7 days unless you disable the safety check.

```
VACUUM table_name [RETAIN n HOURS] [DRY RUN];
```

Example: weekly maintenance notebook.

```
VACUUM lh_silver.sales.orders RETAIN 168 HOURS DRY RUN;  -- list files first
VACUUM lh_silver.sales.orders RETAIN 168 HOURS;
```

> **Trap:** setting `spark.databricks.delta.retentionDurationCheck.enabled=false` and vacuuming with `RETAIN 0 HOURS` can corrupt concurrent readers and long-running streams. Keep at least 7 days.

History and recovery

Time travel

Every commit creates a version. You can query a table as of a version or timestamp, as long as the files have not been vacuumed and the log retention (`delta.logRetentionDuration`, default 30 days) still covers it.

```
SELECT * FROM t VERSION AS OF 42;
SELECT * FROM t TIMESTAMP AS OF '2026-10-25 23:00:00';
```

```
spark.read.option("versionAsOf", 42).table(t)
spark.read.option("timestampAsOf", "2026-10-25").table(t)
```

Example: compare today's gold numbers with yesterday's to find a regression.

```
SELECT cur.store_id, cur.revenue AS today, prev.revenue AS yesterday
FROM lh_gold.gold.fact_store_daily cur
JOIN lh_gold.gold.fact_store_daily TIMESTAMP AS OF '2026-10-26 06:00:00' prev
  ON cur.store_id = prev.store_id AND cur.order_date = prev.order_date
WHERE cur.order_date = '2026-10-25' AND abs(cur.revenue - prev.revenue) > 1;
```

RESTORE

RESTORE rolls a table back to an earlier version as a new commit (history is kept).

```
RESTORE TABLE t TO VERSION AS OF n;
RESTORE TABLE t TO TIMESTAMP AS OF 'ts';
```

Example: a bad deployment overwrote the customer dimension at version 118.

```
DESCRIBE HISTORY lh_gold.gold.dim_customer LIMIT 5;   -- find last good version: 117
RESTORE TABLE lh_gold.gold.dim_customer TO VERSION AS OF 117;
```

Change Data Feed (CDF)

CDF records row-level changes (insert, update_preimage, update_postimage, delete) for each commit, so downstream jobs process only what changed. Enable it per table; changes are kept with the table's file retention.

```
ALTER TABLE t SET TBLPROPERTIES (delta.enableChangeDataFeed = true);
SELECT * FROM table_changes('t', start_version [, end_version]);
```

```
spark.read.option("readChangeFeed", "true").option("startingVersion", v).table(t)
```

Example: propagate silver customer changes to gold incrementally with a stored last-processed version.

```
last_v = int(spark.sql("SELECT max_version FROM ops.cdf_watermarks WHERE
table_name='customers'").first()[0])
changes = (spark.read.option("readChangeFeed", "true").option("startingVersion", last_v + 1)
           .table("lh_silver.customer.customers")
           .where("_change_type IN ('insert','update_postimage','delete')"))
# ... apply to gold dim (see SCD2) ...
new_v = spark.sql("DESCRIBE HISTORY lh_silver.customer.customers LIMIT 1").first()["version"]
spark.sql(f"UPDATE ops.cdf_watermarks SET max_version = {new_v} WHERE table_name='customers'")
```

Schema management

Schema enforcement and evolution

Delta rejects writes whose schema does not match the table (enforcement). You can opt in to evolution: `mergeSchema` adds new columns on append, `overwriteSchema` replaces the schema on overwrite, and `autoMerge` lets MERGE add columns. Column mapping mode `name` allows renaming and dropping columns and column names with spaces.

```
df.write.mode("append").option("mergeSchema", "true").saveAsTable(t)
df.write.mode("overwrite").option("overwriteSchema", "true").saveAsTable(t)
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")
```

```
ALTER TABLE t ADD COLUMNS (loyalty_tier STRING AFTER segment);
ALTER TABLE t SET TBLPROPERTIES ('delta.columnMapping.mode' = 'name');
ALTER TABLE t RENAME COLUMN cust_nm TO customer_name;
ALTER TABLE t DROP COLUMN legacy_flag;
```

Example: the CRM team adds `loyalty_tier`; bronze accepts it automatically, silver adds it deliberately.

```
new = spark.read.json("Files/landing/crm/2026-10-27/*.json")   # has loyalty_tier now
new.write.mode("append").option("mergeSchema",
"true").saveAsTable("lh_bronze.bronze.crm_customers")
spark.sql("ALTER TABLE lh_silver.customer.customers ADD COLUMNS (loyalty_tier STRING)")
```

Trap: column mapping and some newer Delta table features raise the table's reader/writer protocol version. Older readers, or engines that don't support those features yet, may then fail to read the table. Check compatibility with the SQL endpoint and Direct Lake before enabling features on gold tables.

Slowly changing dimensions

SCD Type 1

SCD1 overwrites attributes in place and keeps no history. Use it for corrections and attributes nobody reports on historically.

```
MERGE INTO dim t USING src s ON t.business_key = s.business_key
WHEN MATCHED AND (t.attr1 <> s.attr1 OR ...) THEN UPDATE SET t.attr1 = s.attr1, ...
WHEN NOT MATCHED THEN INSERT ...;
```

Example: product dimension, where name and category corrections overwrite.

```
MERGE INTO lh_gold.gold.dim_product t
USING lh_silver.inventory.products s ON t.product_code = s.product_code
WHEN MATCHED AND (t.product_name <> s.product_name OR t.category <> s.category) THEN
  UPDATE SET t.product_name = s.product_name, t.category = s.category, t.updated_at =
current_timestamp()
WHEN NOT MATCHED THEN
  INSERT (product_code, product_name, category, updated_at)
  VALUES (s.product_code, s.product_name, s.category, current_timestamp());
```

SCD Type 2

SCD2 keeps history: when a tracked attribute changes, close the current row (`valid_to`, `is_current = false`) and insert a new current row. The standard single-MERGE trick unions the source twice: once keyed for the update (to close the old row) and once with a NULL merge key (to force an insert of the new version).

Syntax pattern:

```
MERGE INTO dim t
USING (
  SELECT s.business_key AS merge_key, s.* FROM src s            -- closes/updates
  UNION ALL
  SELECT NULL AS merge_key, s.* FROM src s JOIN dim t
    ON s.business_key = t.business_key AND t.is_current AND <tracked cols differ>  -- new versions
) u
ON t.business_key = u.merge_key AND t.is_current
WHEN MATCHED AND <tracked cols differ> THEN UPDATE SET is_current = false, valid_to = <ts>
WHEN NOT MATCHED THEN INSERT (..., valid_from, valid_to, is_current) VALUES (..., <ts>, NULL,
true);
```

Example: customer dimension tracking segment and city.

```
MERGE INTO lh_gold.gold.dim_customer AS t
USING (
  SELECT s.customer_id AS merge_key, s.customer_id, s.name, s.segment, s.city
  FROM lh_silver.customer.customers s
  UNION ALL
  SELECT NULL AS merge_key, s.customer_id, s.name, s.segment, s.city
  FROM lh_silver.customer.customers s
  JOIN lh_gold.gold.dim_customer d
    ON s.customer_id = d.customer_id AND d.is_current = true
   AND (s.segment <> d.segment OR s.city <> d.city)
) AS u
ON t.customer_id = u.merge_key AND t.is_current = true
WHEN MATCHED AND (t.segment <> u.segment OR t.city <> u.city) THEN
  UPDATE SET t.is_current = false, t.valid_to = current_timestamp()
WHEN MATCHED AND t.name <> u.name THEN            -- type-1 attribute on the current row
  UPDATE SET t.name = u.name
WHEN NOT MATCHED THEN
  INSERT (customer_sk, customer_id, name, segment, city, valid_from, valid_to, is_current)
  VALUES (xxhash64(u.customer_id, current_timestamp()), u.customer_id, u.name, u.segment, u.city,
          current_timestamp(), NULL, true);
```

Best practice: compare a hash of tracked columns (`sha2(concat_ws('||', segment, city), 256)`) instead of listing every column, and handle NULLs with `<=>` (null-safe equals) or `coalesce`. For surrogate keys, Delta identity columns (`GENERATED ALWAYS AS IDENTITY`) work in Spark, but a hash or `monotonically_increasing_id()` offset from `max(sk)` is common. Copy job also offers an SCD Type 2 write option for simple cases.

Identity and generated columns

```
CREATE TABLE gold.dim_store (
  store_sk BIGINT GENERATED ALWAYS AS IDENTITY,
  store_id INT, store_name STRING,
  opened_year INT GENERATED ALWAYS AS (year(opened_on)), opened_on DATE) USING DELTA;
```

Example: insert without supplying the surrogate key.

```
INSERT INTO lh_gold.gold.dim_store (store_id, store_name, opened_on)
SELECT store_id, store_name, opened_on FROM lh_silver.inventory.stores;
```

Data Warehouse

Warehouse architecture

How Fabric Warehouse works

Fabric Data Warehouse is a fully transactional T-SQL warehouse whose tables are stored as Delta/Parquet in OneLake. Compute is serverless and distributed (the Polaris engine); there are no distributions, indexes or resource classes to manage. It supports multi-table transactions, and every table is readable by Spark and Direct Lake through OneLake. Versus the lakehouse SQL analytics endpoint, the warehouse lets you write with T-SQL.

Feature	Warehouse	Lakehouse SQL analytics endpoint
T-SQL DML (INSERT/UPDATE/DELETE/MERGE)	Yes	No (read-only)
Write with Spark	No (read only via OneLake)	Yes
Multi-table transactions	Yes	No
Views, functions, procedures	Yes	Yes
Security (GRANT, RLS, CLS, DDM)	Yes	Yes
Schema evolution	ALTER TABLE ADD/DROP column	Follows Delta

```
Workspace > New item > Warehouse > "wh_sales"
Connection string: <unique>.datawarehouse.fabric.microsoft.com   (TDS, port 1433, Entra auth)
```

Example: connect from SSMS or sqlcmd with Entra ID.

```
sqlcmd -S abc123xyz.datawarehouse.fabric.microsoft.com -d wh_sales -G \
  -Q "SELECT TOP 5 name, create_date FROM sys.tables ORDER BY create_date DESC"
```

Custom SQL pools and workload isolation

The warehouse separates **SELECT** (read) and **ingestion** workloads onto different compute so loads don't slow reports. Custom SQL pools (GA September 2026) let admins define named pools with their own resource share and route workloads by application name or similar rules. Preview items announced at FabCon Europe 2026 include GPU-accelerated query execution and node-based billing.

```
Warehouse settings > SQL pools > + New pool "etl_pool" (40%)  "bi_pool" (60%)
Classifier: application name LIKE 'PowerBI%' → bi_pool ; 'Pipeline%' → etl_pool
```

Example: tag a pipeline's connection so it lands in the ETL pool (check the classifier options your tenant shows).

```
Script activity connection > Additional properties: Application Name = Pipeline-pl_daily_sales
```

T-SQL DDL

Tables and data types

Fabric Warehouse supports a subset of SQL Server types. Notable gaps: no `datetime` (use `datetime2(6)`), no `nvarchar` historically (use `varchar` with UTF-8 collation; check current docs, as support evolves), no `money`, `xml`, `geography`, or `text`. Constraints PRIMARY KEY, UNIQUE and FOREIGN KEY must be `NOT ENFORCED`.

Use	Type
Integers	`int`, `bigint`, `smallint`, `bit`
Money	`decimal(p,s)`
Text	`varchar(n)`, `char(n)` (UTF-8)
Dates	`date`, `time(6)`, `datetime2(6)`
Other	`float`, `real`, `varbinary(n)`, `uniqueidentifier`

Syntax:

```
CREATE TABLE schema.table (
  col type [NOT NULL] [IDENTITY], ...
);
ALTER TABLE schema.table ADD CONSTRAINT pk PRIMARY KEY NONCLUSTERED (col) NOT ENFORCED;
```

Example: star schema for sales.

```
CREATE SCHEMA sales;
GO
CREATE TABLE sales.dim_customer (
  customer_sk  BIGINT IDENTITY,            -- auto-generated, not guaranteed contiguous
  customer_id  BIGINT       NOT NULL,
  name         VARCHAR(200) NOT NULL,
  segment      VARCHAR(50),
  city         VARCHAR(100),
  valid_from   DATETIME2(6) NOT NULL,
  valid_to     DATETIME2(6) NULL,
  is_current   BIT          NOT NULL
);
CREATE TABLE sales.fact_sales (
  order_id     BIGINT NOT NULL,
  order_date   DATE   NOT NULL,
  customer_sk  BIGINT NOT NULL,
  product_sk   BIGINT NOT NULL,
  store_id     INT    NOT NULL,
  quantity     INT,
  net_amount   DECIMAL(18,2)
);
ALTER TABLE sales.dim_customer ADD CONSTRAINT pk_dim_customer
  PRIMARY KEY NONCLUSTERED (customer_sk) NOT ENFORCED;
```

Trap: IDENTITY in Fabric Warehouse is `BIGINT IDENTITY` only, with no custom seed or increment, and values can have gaps. Don't use it where gap-free sequences are a business rule (invoice numbers).

Views, functions, stored procedures

```
CREATE OR ALTER VIEW s.v AS SELECT ...;
CREATE OR ALTER FUNCTION s.f(@p INT) RETURNS TABLE AS RETURN (SELECT ...);   -- inline TVF
CREATE OR ALTER PROCEDURE s.p @p DATE AS BEGIN ... END;
```

Example: an ETL procedure with a transaction and error handling.

```
CREATE OR ALTER PROCEDURE sales.usp_load_fact_sales @load_date DATE
AS
BEGIN
  SET NOCOUNT ON;
  BEGIN TRY
    BEGIN TRANSACTION;
      DELETE FROM sales.fact_sales WHERE order_date = @load_date;
      INSERT INTO sales.fact_sales (order_id, order_date, customer_sk, product_sk, store_id,
quantity, net_amount)
      SELECT l.order_id, l.order_date, c.customer_sk, p.product_sk, l.store_id, l.quantity,
l.net_amount
      FROM   lh_silver.sales.order_lines AS l                -- cross-database read from lakehouse
      JOIN   sales.dim_customer AS c ON c.customer_id = l.customer_id AND c.is_current = 1
      JOIN   sales.dim_product  AS p ON p.product_code = l.product_code
      WHERE  l.order_date = @load_date;
    COMMIT TRANSACTION;
  END TRY
  BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
  END CATCH
END;
GO
EXEC sales.usp_load_fact_sales @load_date = '2026-10-26';
```

Loading data

COPY INTO

COPY INTO is the fastest way to bulk load Parquet or CSV from Azure storage or OneLake into a warehouse table. It supports wildcards, column lists, error handling and Entra/SAS credentials.

```
COPY INTO schema.table [(col_list)]
FROM 'https://<account>.dfs.core.windows.net/<container>/<path>/*.parquet'
WITH (
  FILE_TYPE = 'PARQUET' | 'CSV',
  CREDENTIAL = (IDENTITY = 'Shared Access Signature', SECRET = '<sas>'),
  FIRSTROW = 2, FIELDTERMINATOR = ',', ROWTERMINATOR = '0x0A',
  MAXERRORS = n, ERRORFILE = '<path>'
);
```

Example: load a day of POS Parquet files from a lakehouse Files folder.

```
COPY INTO sales.stg_pos_lines
FROM 'https://onelake.dfs.fabric.microsoft.com/ws-retail-
prod/lh_bronze.Lakehouse/Files/landing/pos/2026/10/26/*.parquet'
WITH (FILE_TYPE = 'PARQUET');

COPY INTO sales.stg_returns (return_id, order_id, amount, reason)
FROM 'https://contosodatalake.dfs.core.windows.net/raw/returns/2026-10-26/*.csv'
WITH (FILE_TYPE = 'CSV', FIRSTROW = 2, FIELDTERMINATOR = ';',
      CREDENTIAL = (IDENTITY = 'Shared Access Signature', SECRET = 'sv=2025-...'),
      MAXERRORS = 10, ERRORFILE = 'https://contosodatalake.dfs.core.windows.net/raw/_errors/');
```

CTAS, INSERT...SELECT, SELECT INTO

```
CREATE TABLE s.new AS SELECT ...;          -- CTAS: create + load in one statement
INSERT INTO s.t (cols) SELECT ...;
SELECT ... INTO s.new FROM ...;
```

Example: rebuild an aggregate table with CTAS and swap it in.

```
CREATE TABLE sales.agg_store_month_new AS
SELECT store_id, DATETRUNC(month, order_date) AS month_start,
       SUM(net_amount) AS revenue, COUNT_BIG(*) AS lines
FROM sales.fact_sales
GROUP BY store_id, DATETRUNC(month, order_date);

BEGIN TRANSACTION;
  DROP TABLE IF EXISTS sales.agg_store_month;
  EXEC sp_rename 'sales.agg_store_month_new', 'agg_store_month';
COMMIT;
```

OPENROWSET and ingestion from files

OPENROWSET reads Parquet, CSV and JSONL files directly (from OneLake or Azure storage) in a query, which is useful for exploration and for INSERT...SELECT with transformations.

```
SELECT * FROM OPENROWSET(BULK '<url>', FORMAT = 'parquet' | 'csv' [, HEADER_ROW = TRUE]) AS r;
```

Example: preview a file before loading.

```
SELECT TOP 20 *
FROM OPENROWSET(
  BULK 'https://onelake.dfs.fabric.microsoft.com/ws-retail-
prod/lh_bronze.Lakehouse/Files/landing/stores.csv',
  FORMAT = 'csv', HEADER_ROW = TRUE) AS stores;
```

MERGE in the warehouse

MERGE is documented for Fabric Data Warehouse (it was missing in early releases; verify current limitations in the T-SQL surface-area page). Before MERGE, the standard pattern was UPDATE + INSERT in a transaction, which you will still see in older code.

```
MERGE sales.dim_product AS t
USING sales.stg_products AS s ON t.product_code = s.product_code
WHEN MATCHED AND t.product_name <> s.product_name THEN UPDATE SET product_name = s.product_name
WHEN NOT MATCHED BY TARGET THEN INSERT (product_code, product_name, category)
                               VALUES (s.product_code, s.product_name, s.category);
```

Example: the older two-step pattern (portable, works everywhere).

```
BEGIN TRANSACTION;
UPDATE t SET t.product_name = s.product_name, t.category = s.category
FROM sales.dim_product t JOIN sales.stg_products s ON t.product_code = s.product_code
WHERE t.product_name <> s.product_name OR t.category <> s.category;

INSERT INTO sales.dim_product (product_code, product_name, category)
SELECT s.product_code, s.product_name, s.category
FROM sales.stg_products s
WHERE NOT EXISTS (SELECT 1 FROM sales.dim_product t WHERE t.product_code = s.product_code);
COMMIT;
```

Querying across items

Cross-database queries

Within one workspace you can join warehouse tables with lakehouse SQL endpoints, mirrored databases and other warehouses using three-part names. Add items from other workspaces via shortcuts.

```
SELECT ... FROM [database].[schema].[table] a JOIN [other_db].[schema].[table] b ON ...;
```

Example: enrich warehouse facts with a lakehouse table and a mirrored CRM table.

```
SELECT f.order_date, s.region, crm.account_manager, SUM(f.net_amount) AS revenue
FROM   wh_sales.sales.fact_sales      AS f
JOIN   lh_silver.inventory.stores     AS s   ON s.store_id = f.store_id
JOIN   mirror_crm.dbo.accounts        AS crm ON crm.store_id = f.store_id
WHERE  f.order_date >= '2026-10-01'
GROUP BY f.order_date, s.region, crm.account_manager;
```

Security in the warehouse

Object-level security (GRANT/DENY)

Workspace roles give broad access. For finer control, share the warehouse with only Read (connect) and grant object permissions with T-SQL.

```
GRANT SELECT ON SCHEMA::sales TO [analysts@contoso.com];
DENY SELECT ON sales.fact_payroll TO [analysts@contoso.com];
GRANT EXECUTE ON sales.usp_load_fact_sales TO [sp-fabric-etl];
```

Example: create a role for store managers.

```
CREATE ROLE store_managers;
ALTER ROLE store_managers ADD MEMBER [sg-store-managers@contoso.com];
GRANT SELECT ON sales.fact_sales   TO store_managers;
GRANT SELECT ON sales.dim_customer TO store_managers;
```

Row-level security (RLS)

RLS filters rows with a predicate function bound by a security policy. Users only see rows the function returns.

```
CREATE FUNCTION sec.fn_filter(@col type) RETURNS TABLE WITH SCHEMABINDING AS
  RETURN SELECT 1 AS ok WHERE <condition using USER_NAME()/SESSION_CONTEXT>;
CREATE SECURITY POLICY sec.policy ADD FILTER PREDICATE sec.fn_filter(col) ON schema.table WITH
(STATE = ON);
```

Example: managers see only their stores, mapped in a security table.

```
CREATE SCHEMA sec;
GO
CREATE TABLE sec.store_access (user_upn VARCHAR(200), store_id INT);
GO
CREATE FUNCTION sec.fn_store_filter(@store_id INT)
RETURNS TABLE WITH SCHEMABINDING AS
RETURN SELECT 1 AS allowed
       FROM sec.store_access a
       WHERE a.store_id = @store_id AND a.user_upn = USER_NAME()
       UNION ALL
       SELECT 1 WHERE IS_ROLEMEMBER('db_owner') = 1;   -- admins see everything
GO
CREATE SECURITY POLICY sec.store_policy
  ADD FILTER PREDICATE sec.fn_store_filter(store_id) ON sales.fact_sales
  WITH (STATE = ON);
```

Column-level security (CLS)

CLS grants SELECT on specific columns only. Querying a denied column (or `SELECT *`) fails.

```
GRANT SELECT ON schema.table (col1, col2) TO principal;
```

Example: analysts can't see email or phone.

```
GRANT SELECT ON sales.dim_customer (customer_sk, customer_id, name, segment, city) TO
[analysts@contoso.com];
```

Dynamic data masking (DDM)

DDM masks values in query results for users without UNMASK permission. Data is not encrypted; it is obfuscated at query time, so treat it as a convenience, not a security boundary against users who can run ad hoc queries.

```
ALTER TABLE t ALTER COLUMN c ADD MASKED WITH (FUNCTION = 'default()' | 'email()' |
'partial(p,"xxx",s)' | 'random(a,b)');
GRANT UNMASK [ON schema.table] TO principal;
```

Example:

```
ALTER TABLE sales.dim_customer ALTER COLUMN email ADD MASKED WITH (FUNCTION = 'email()');
ALTER TABLE sales.dim_customer ALTER COLUMN phone ADD MASKED WITH (FUNCTION = 'partial(0,"XXX-XXX-
",4)');
GRANT UNMASK ON sales.dim_customer TO [sg-customer-service@contoso.com];
```

> **Trap:** SQL-level RLS/CLS/DDM apply only to queries through the SQL engine. A user with workspace Contributor (or ReadAll on OneLake) can read the Parquet files directly with Spark. Use OneLake security or restrict workspace roles to close that path.

Statistics, recovery and snapshots

Statistics

The warehouse creates and updates statistics automatically (histogram stats on columns used in queries, plus proactive refresh). You can create and update them manually for columns with skewed data or after large loads.

```
CREATE STATISTICS stat_name ON schema.table (col) [WITH FULLSCAN | SAMPLE n PERCENT];
UPDATE STATISTICS schema.table (stat_name) [WITH FULLSCAN];
```

Example: after a backfill of 2 billion rows.

```
CREATE STATISTICS st_fact_sales_customer ON sales.fact_sales (customer_sk) WITH FULLSCAN;
UPDATE STATISTICS sales.fact_sales;
SELECT name, auto_created, user_created FROM sys.stats WHERE object_id =
OBJECT_ID('sales.fact_sales');
```

Zero-copy clone

Cloning creates a new table that points to the same Parquet files as the source at a point in time. Only metadata is copied; later changes to either table are independent. Clones can be taken as of a past point within the retention period.

```
CREATE TABLE schema.new_table AS CLONE OF schema.source_table [AT 'yyyy-MM-ddTHH:mm:ss.fff'];
```

Example: snapshot a dimension before a risky change, and create a test copy as of last night.

```
CREATE TABLE sales.dim_customer_backup_20261027 AS CLONE OF sales.dim_customer;
CREATE TABLE sales.fact_sales_asof AS CLONE OF sales.fact_sales AT '2026-10-26T23:00:00.000';
```

Time travel in T-SQL

Query a warehouse table as it was at a point in time (UTC) within the retention window (30 days by default) with a query hint.

```
SELECT ... FROM t OPTION (FOR TIMESTAMP AS OF 'yyyy-MM-ddTHH:mm:ss.fff');
```

Example: reproduce last month-end numbers for audit.

```
SELECT store_id, SUM(net_amount) AS revenue
FROM sales.fact_sales
WHERE order_date BETWEEN '2026-09-01' AND '2026-09-30'
GROUP BY store_id
OPTION (FOR TIMESTAMP AS OF '2026-10-01T06:00:00.000');
```

Warehouse snapshots and restore points

A **warehouse snapshot** is a read-only item that shows the warehouse at a point in time; you roll it forward after ETL completes, so reports never see half-loaded data. **Restore points** (system every 8 hours, or user-defined) let you restore the whole warehouse in place.

```
Warehouse > New warehouse snapshot "wh_sales_snapshot"   (read-only, own SQL endpoint)
ETL done → Snapshot settings > Update timestamp to current
Warehouse > Settings > Restore points > + Create restore point "before_v2_release"
```

Example: point the semantic model at `wh_sales_snapshot`, and add a final pipeline step that advances the snapshot after the load passes validation.

```
{ "name": "Advance snapshot", "type": "WebActivity",
  "typeProperties": { "method": "PATCH",
    "relativeUrl": "/v1/workspaces/@{pipeline().DataFactory}/warehousesnapshots/<snapshotId>",
    "body": { "properties": { "snapshotDateTime": "@{utcNow()}" } } } }
```

(The exact REST path and body shape for warehouse snapshots should be checked in the current API reference; the pattern is a PATCH that updates the snapshot timestamp.)

Monitoring the warehouse

Query insights and DMVs

The `queryinsights` schema keeps 30 days of completed query history, long-running and frequently run queries. DMVs show live sessions and requests.

```
SELECT * FROM queryinsights.exec_requests_history;
SELECT * FROM queryinsights.long_running_queries;
SELECT * FROM queryinsights.frequently_run_queries;
SELECT * FROM sys.dm_exec_requests;  KILL <session_id>;
```

Example: find the slowest queries of the last day and who ran them.

```
SELECT TOP 10 start_time, login_name, total_elapsed_time_ms, row_count,
       data_scanned_remote_storage_mb, LEFT(command, 200) AS query_text
FROM queryinsights.exec_requests_history
WHERE start_time >= DATEADD(day, -1, SYSUTCDATETIME())
ORDER BY total_elapsed_time_ms DESC;
```

Result set caching and other features

Result set caching stores results of repeated SELECTs for reuse; data clustering and distributed bitmap filters (GA September 2026) speed up selective scans and star joins. Recent previews add Teradata migration help and new analytical functions.

```
ALTER DATABASE wh_sales SET RESULT_SET_CACHING ON;
```

Example: a dashboard query that runs every minute against unchanged data is answered from the cache, cutting both latency and CU use. Check `result_cache_hit` in `queryinsights.exec_requests_history`.

```
SELECT TOP 20 start_time, result_cache_hit, total_elapsed_time_ms
FROM queryinsights.exec_requests_history
WHERE command LIKE '%agg_store_month%' ORDER BY start_time DESC;
```

Data Factory: pipelines, Copy job and Dataflows Gen2

Choosing an ingestion tool

Tool	Code	Transform	Best for	Gotcha
Copy job	None	Light (mapping)	Bulk, incremental and CDC copies from many sources, many tables	Less control over orchestration
Pipeline Copy activity	Low (JSON/expressions)	Light	Copy inside an orchestrated flow	Incremental logic is yours to build
Dataflow Gen2	Low (Power Query M)	Rich, 300+ transforms	Business-friendly transforms, small/medium volume	Higher CU per GB than Spark at scale
Notebook	Code	Anything	Large volume, complex logic	Needs Spark skills
Mirroring	None	None	Near real-time replica of a database	Supported sources only
Eventstream	None/SQL	Stream ops	Events, streaming	Event-shaped data

Pipelines

Pipeline structure and activities

A pipeline is a JSON-defined workflow of activities linked by dependency conditions (Succeeded, Failed, Completed, Skipped). It is the Fabric version of Azure Data Factory pipelines.

Category	Activities
Data movement	Copy data, Copy job, Invoke pipeline
Transform	Notebook, Spark job definition, Dataflow, Stored procedure, Script, KQL, Semantic model refresh, Azure Databricks, Azure Functions
Control flow	ForEach, If Condition, Switch, Until, Wait, Set variable, Append variable, Filter, Fail, Lookup, Get metadata
External	Web, Webhook, Office 365 Outlook, Teams

Syntax (activity JSON skeleton):

```
{
  "name": "Activity name", "type": "Copy | TridentNotebook | Lookup | ForEach | ...",
  "dependsOn": [ { "activity": "Previous", "dependencyConditions": [ "Succeeded" ] } ],
  "policy": { "timeout": "0.12:00:00", "retry": 2, "retryIntervalInSeconds": 60 },
  "typeProperties": { }
}
```

Example: `pl_daily_sales` outline.

```
pl_daily_sales (params: run_date)
 1 Lookup  "Get tables"        → SELECT * FROM ops.load_config WHERE enabled = 1   (SQL database)
 2 ForEach "Each table" (batch 8, sequential=false)
     2a Copy "Copy to bronze"  (source query uses watermark; sink lh_bronze Files/Tables)
     2b Stored procedure "Update watermark"  (on Succeeded)
 3 Notebook "Silver orders"    (session tag silver_batch)
 4 Notebook "Gold facts"
 5 Semantic model refresh "sm_sales"  (or none if Direct Lake auto-updates)
 6 Teams "Notify failure"      (on Failed of 3 or 4)
```

Parameters, variables and expressions

Pipeline **parameters** are set at run time and are read-only. **Variables** can change during a run (Set/Append variable). **Expressions** use the `@` syntax with functions such as `concat`, `formatDateTime`, `addDays`, `utcNow`, `coalesce`, `json`, `item()`, `activity('x').output`. Library variables come from a **Variable library** item and change per deployment stage.

```
@pipeline().parameters.run_date
@variables('row_count')
@activity('Get tables').output.value
@item().table_name                          (inside ForEach)
@concat('Files/landing/', item().table_name, '/', formatDateTime(utcNow(), 'yyyy/MM/dd'))
@if(equals(pipeline().parameters.full_load, true), '1900-01-01', activity('Get
WM').output.firstRow.wm)
@pipeline().libraryVariables.source_server   (Variable library reference)
@pipeline().RunId   @pipeline().TriggerTime   @pipeline().DataFactory (workspace id)
```

Example: a dynamic source query in a Copy activity.

```
"source": {
  "type": "AzureSqlSource",
  "sqlReaderQuery": {
    "value": "SELECT * FROM @{item().schema_name}.@{item().table_name} WHERE
@{item().watermark_column} > '@{item().last_watermark}' AND @{item().watermark_column} <=
'@{pipeline().TriggerTime}'",
    "type": "Expression"
  }
}
```

Incremental load with a watermark

The classic pattern stores the last loaded value of a monotonically increasing column (modified date or rowversion) in a control table. Each run reads rows greater than the old watermark and up to a new high value captured at the start, then updates the watermark only if the copy succeeded.

```
1 Lookup old watermark       SELECT last_wm FROM ops.watermarks WHERE table_name = 'orders'
2 Lookup new watermark       SELECT MAX(modified_at) AS new_wm FROM src.dbo.orders
3 Copy  WHERE modified_at > old AND modified_at <= new
4 Stored proc  UPDATE ops.watermarks SET last_wm = new WHERE table_name = 'orders'
```

Example: control table and update procedure in a Fabric SQL database or warehouse.

```
CREATE TABLE ops.watermarks (
  table_name VARCHAR(128) NOT NULL, watermark_column VARCHAR(128) NOT NULL,
  last_wm DATETIME2(6) NOT NULL, updated_at DATETIME2(6) NOT NULL);
INSERT INTO ops.watermarks VALUES ('orders', 'modified_at', '1900-01-01', SYSUTCDATETIME());
GO
CREATE OR ALTER PROCEDURE ops.usp_set_watermark @table_name VARCHAR(128), @new_wm DATETIME2(6)
AS
UPDATE ops.watermarks SET last_wm = @new_wm, updated_at = SYSUTCDATETIME()
WHERE table_name = @table_name;
```

```
{ "name": "Copy orders incremental", "type": "Copy",
  "dependsOn": [{"activity": "Get new WM", "dependencyConditions": ["Succeeded"]}],
  "typeProperties": {
    "source": { "type": "AzureSqlSource", "sqlReaderQuery": {
      "value": "SELECT * FROM dbo.orders WHERE modified_at > '@{activity('Get old
WM').output.firstRow.last_wm}' AND modified_at <= '@{activity('Get new
WM').output.firstRow.new_wm}'",
      "type": "Expression" } },
    "sink": { "type": "LakehouseTableSink", "tableActionOption": "Append" } } }
```

Trap: capture the new watermark *before* copying and use `<=` it, not `utcNow()` after the copy. Otherwise rows committed during the copy are skipped. Watermarks on `modified_at` also miss hard deletes; use CDC, change tracking or periodic full reconciles for deletes.

Other incremental patterns

Pattern	Detects deletes	Needs	Fabric tool
Watermark column	No	Reliable modified timestamp	Pipeline, Copy job (incremental)
SQL Server CDC / change tracking	Yes	CDC enabled at source	Copy job CDC, mirroring
Delta CDF	Yes	Delta source	Notebook, Copy job
File arrival (new files only)	n/a	Folder by date or event trigger	Pipeline + storage event trigger
Full extract + hash compare	Yes (anti-join)	Small/medium tables	Notebook MERGE with NOT MATCHED BY SOURCE

```
Copy job > Copy mode: Incremental copy > per table: incremental column = modified_at
          (or CDC for CDC-enabled sources) > Update method: Append | Merge (keys) | SCD Type 2
```

Example: file-arrival processing with a OneLake/storage event trigger.

```
pl_ingest_returns > Add trigger > Azure Blob Storage events (or OneLake events)
  Event: Microsoft.Storage.BlobCreated  Path begins with: /raw/returns/
  Pipeline parameter file_name = @triggerBody().subject   (pass the file path)
```

Control flow: ForEach, If, Until, Switch

```
{ "type": "ForEach", "typeProperties": { "items": "@activity('Get tables').output.value",
  "isSequential": false, "batchCount": 8, "activities": [ ... ] } }
{ "type": "IfCondition", "typeProperties": { "expression": "@greater(int(variables('rows')), 0)",
  "ifTrueActivities": [ ... ], "ifFalseActivities": [ ... ] } }
{ "type": "Until", "typeProperties": { "expression": "@equals(variables('status'), 'Done')",
"timeout": "0.01:00:00", "activities": [ ... ] } }
```

Example: skip the silver step when bronze received zero rows, and fail loudly with a custom message on bad data.

```
{ "name": "Any rows?", "type": "IfCondition",
  "typeProperties": {
    "expression": "@greater(activity('Copy orders incremental').output.rowsCopied, 0)",
    "ifTrueActivities": [ { "name": "Silver orders", "type": "TridentNotebook", "typeProperties": {
"notebookId": "..." } } ],
    "ifFalseActivities": [ { "name": "Log no data", "type": "SetVariable",
        "typeProperties": { "variableName": "status", "value": "no-new-rows" } } ] } },
{ "name": "Fail on bad DQ", "type": "Fail",
  "typeProperties": { "message": "@concat('DQ failed: ', activity('DQ
check').output.result.exitValue)", "errorCode": "DQ001" } }
```

Scheduling, triggers and retries

Pipelines can run on schedules (several schedules per item), on events (storage/OneLake events via Real-Time hub, Activator rules) or on demand via REST. Use activity retry policies for transient errors and a top-level failure path for alerts.

```
POST https://api.fabric.microsoft.com/v1/workspaces/{wsId}/items/{pipelineId}/jobs/instances?
jobType=Pipeline
{ "executionData": { "parameters": { "run_date": "2026-10-26", "full_load": false } } }
```

Example: trigger from an Azure DevOps release, then poll status.

```
loc=$(curl -si -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"executionData":{"parameters":{"run_date":"2026-10-26"}}}' \
  "https://api.fabric.microsoft.com/v1/workspaces/$WS/items/$PL/jobs/instances?jobType=Pipeline" \
  | grep -i '^location:' | awk '{print $2}' | tr -d '\r')
curl -s -H "Authorization: Bearer $TOKEN" "$loc" | jq '.status'   # NotStarted | InProgress |
Completed | Failed
```

Connections and the on-premises data gateway

Connections store credentials centrally (Manage connections and gateways) and are shared by pipelines, Copy jobs and dataflows. On-prem or VNet-locked sources need an **on-premises data gateway** or a **VNet data gateway**. Credentials can come from Azure Key Vault references.

```
Settings (gear) > Manage connections and gateways > + New
  Gateway: gw-onprem-cluster (2 nodes)   Type: SQL Server   Server: sql01.contoso.local
  Auth: Basic / Windows / Key Vault reference   Privacy: Organizational
```

Example: a Copy activity from on-prem SQL Server through the gateway into the lakehouse; the activity just references the connection ID, and the gateway handles network access.

```
"source": { "type": "SqlServerSource", "datasetSettings": { "externalReferences": { "connection":
"c0ffee00-...-onpremsql" },
            "typeProperties": { "schema": "dbo", "table": "orders" } } }
```

Variable libraries

A variable library item holds named variables with a default value set plus alternative value sets per stage (dev, test, prod). Pipelines, notebooks, Copy jobs, dataflows and shortcuts can reference them, and deployment pipelines or fabric-cicd pick the active value set per workspace. This replaces hard-coded server names and IDs.

```
{
  "variables": [
    { "name": "src_sql_server", "type": "String", "value": "sql-dev.contoso.com" },
    { "name": "lakehouse_id",   "type": "String", "value": "7d1c-dev-..." },
    { "name": "batch_size",     "type": "Integer", "value": 4 }
  ],
  "valueSets": {
    "Test": { "src_sql_server": "sql-test.contoso.com", "lakehouse_id": "7d1c-test-..." },
    "Prod": { "src_sql_server": "sql-prod.contoso.com", "lakehouse_id": "7d1c-prod-...",
"batch_size": 16 }
  }
}
```

Example: in a notebook, read a variable at run time.

```
vl = notebookutils.variableLibrary.getLibrary("vl_retail")
print(vl.src_sql_server)        # sql-prod.contoso.com in the prod workspace
```

Copy job

Copy job modes

Copy job is a dedicated item for moving data without building a pipeline. It supports full copy, incremental copy (watermark column chosen per table), CDC from CDC-enabled sources, and update methods append, overwrite, merge (upsert on keys) and, as of 2026, SCD Type 2. It can copy many tables at once, run on a schedule, use Variable library connections and can be called from a pipeline with the Copy job activity.

```
New item > Copy job > Source: Azure SQL DB (conn) > select tables: dbo.orders, dbo.customers
  > Destination: lh_bronze (Tables) > Copy mode: Incremental (watermark = modified_at)
  > Update method: Merge (keys: order_id) > Schedule: every 30 min > Save + Run
```

Example: the Copy job definition as stored in Git (shape simplified).

```
{
  "properties": {
    "jobMode": "Batch",
    "source": { "type": "AzureSqlDatabase", "connection": "@library('vl_retail').crm_conn" },
    "destination": { "type": "Lakehouse", "workspaceId": "...", "itemId": "..." },
    "policy": { "timeout": "0.12:00:00" }
  },
  "activities": [
    { "source": { "table": "dbo.orders" }, "destination": { "table": "bronze.orders" },
      "copyMode": "Incremental", "watermarkColumn": "modified_at",
      "writeBehavior": "Merge", "keyColumns": ["order_id"] }
  ]
}
```

(The JSON shape above is illustrative; generate the real definition from the UI and keep it in Git.)

Dataflows Gen2

Power Query transformations

Dataflows Gen2 use Power Query (M) with a visual editor and write to destinations (lakehouse, warehouse, SQL database, Azure SQL, Kusto) with append or replace. They use staging (a hidden lakehouse/warehouse) and a "fast copy" path for large simple loads. Dataflows Gen2 with CI/CD support Git, public parameters for pipeline-passed values and Variable library references.

```
let
  Source  = Sql.Database("sql-prod.contoso.com", "crm"),
  Cust    = Source{[Schema="dbo", Item="customers"]}[Data],
  Typed   = Table.TransformColumnTypes(Cust, {{"created_at", type datetime}, {"segment", type
text}}),
  Clean   = Table.SelectRows(Typed, each [email] <> null and Text.Contains([email], "@")),
  Upper   = Table.TransformColumns(Clean, {{"segment", Text.Upper, type text}})
in
  Upper
```

Example: incremental refresh in Dataflow Gen2 (built-in setting) for a large orders query.

```
Query "orders" > Incremental refresh
  Filter column: order_date   Extract data from the past: 3 months
  Bucket size: Day   Only extract new data when max value changes: on
  Destination: lh_bronze.bronze.orders_df (Append)
```

Best practice: keep Dataflows Gen2 for sources and transforms Power Query handles well (SharePoint, Excel, APIs with pagination, small SaaS sources). For tens of millions of rows, a Copy job plus a notebook is usually cheaper in CU.

Apache Airflow job

The Apache Airflow job item gives a managed Airflow environment for teams that orchestrate in Python DAGs, with operators to run Fabric items.

```
from airflow import DAG
from airflow.utils.dates import days_ago
from apache_airflow_microsoft_fabric_plugin.operators.fabric import FabricRunItemOperator

with DAG("retail_nightly", start_date=days_ago(1), schedule="0 2 * * *", catchup=False) as dag:
    silver = FabricRunItemOperator(task_id="silver", fabric_conn_id="fabric_spn",
              workspace_id="b2c1...", item_id="nb-silver-id", job_type="RunNotebook",
wait_for_termination=True)
    gold = FabricRunItemOperator(task_id="gold", fabric_conn_id="fabric_spn",
              workspace_id="b2c1...", item_id="pl-gold-id", job_type="Pipeline",
wait_for_termination=True)
    silver >> gold
```

Example use: a team migrating from on-prem Airflow keeps its DAGs and only swaps operators to call Fabric notebooks and pipelines.

```
Workspace > New item > Apache Airflow job > Sync DAGs from Git repo (dags/) > Run DAG
retail_nightly
```

Real-Time Intelligence

RTI building blocks

Item	What it does	Analogy
Real-Time hub	Catalog of streams, events and Fabric/Azure event sources in the tenant	Discovery
Eventstream	Capture, transform and route events, no-code or SQL	Kafka + stream processor
Eventhouse	Container of KQL databases with shared compute	Azure Data Explorer cluster
KQL database	Tables, functions, update policies, materialized views	ADX database
KQL queryset	Saved KQL (and T-SQL) queries	SSMS query window
Real-Time dashboard	Tiles on KQL queries with auto refresh	Live dashboard
Activator	Rules on streams or data that fire actions (email, Teams, pipelines, notebooks, Power Automate)	Alerting/automation engine

```
IoT Hub / Event Hubs / Kafka / CDC (Debezium) / custom app
        |
   Eventstream es_telemetry ── filter, manage fields, aggregate, SQL operator
        ├──► Eventhouse eh_telemetry / KQL DB kql_iot (table RawReadings)
        ├──► Lakehouse lh_bronze (Delta, for history)
        └──► Activator act_alerts (temperature > 8°C for 5 min → Teams + pipeline)
```

Eventstreams

Sources, destinations and processing

Eventstreams ingest from Azure Event Hubs, IoT Hub, Kafka (Confluent, Amazon MSK), database CDC (Azure SQL, PostgreSQL, MySQL, Cosmos DB), Google Pub/Sub, Amazon Kinesis, Fabric workspace/OneLake/job events, Azure blob events, sample data and a custom endpoint (Event Hubs/Kafka/AMQP compatible). Destinations are Eventhouse (direct or with processing), Lakehouse, Activator, derived streams and custom endpoints. Event processing operators include filter, manage fields, aggregate, group by (tumbling, hopping, sliding, session, snapshot windows), join, union and expand; the SQL operator (GA 2026) lets you write stream queries in SQL and fan out to several destinations.

Ingestion mode to Eventhouse	Processing	When
Direct ingestion	None in Eventstream; use update policies in KQL	High volume, transforms in KQL
Event processing before ingestion	Eventstream operators	Filtering/aggregating before storage

Syntax (custom endpoint, send with the Event Hubs SDK):

```python
from azure.eventhub import EventHubProducerClient, EventData
import json
producer = EventHubProducerClient.from_connection_string(conn_str="<Eventstream custom endpoint connection string>")
batch = producer.create_batch()
batch.add(EventData(json.dumps({"device_id": "fridge-0412", "temp_c": 7.9, "ts": "2026-10-27T09:15:02Z"})))
producer.send_batch(batch)
```

Example: Eventstream SQL operator that filters bad readings and computes 1-minute averages per store.

```
SELECT
    store_id,
    device_id,
    AVG(temp_c)      AS avg_temp_c,
    MAX(temp_c)      AS max_temp_c,
    System.Timestamp() AS window_end
INTO  [kql-iot-readings-1m]                -- output (destination) name
FROM  [es-telemetry-input] TIMESTAMP BY ts  -- event time
WHERE temp_c BETWEEN -40 AND 60
GROUP BY store_id, device_id, TumblingWindow(minute, 1);
```

Eventhouse and KQL databases

Eventhouse fundamentals

An Eventhouse hosts KQL databases with shared, autoscaling compute. Data is stored in compressed columnar extents with automatic indexing on all columns, so time-range and text filters are fast. Two policies control data life: **retention** (how long data is kept) and **caching** (how long it stays on hot SSD cache). **OneLake availability** writes KQL tables to OneLake as Delta so Spark and Power BI can read them. **Minimum consumption** keeps compute always on for low-latency workloads at a cost.

```
.create table RawReadings (device_id: string, store_id: int, temp_c: real, humidity: real, ts:
datetime)
.alter table RawReadings policy retention '{"SoftDeletePeriod": "365.00:00:00", "Recoverability":
"Enabled"}'
.alter table RawReadings policy caching hot = 30d
.alter-merge table RawReadings policy mirroring dataformat=parquet with (IsEnabled=true)   //
OneLake availability
```

Example: ingest a sample file for testing and check the table.

```
.ingest into table RawReadings
('https://contosodatalake.blob.core.windows.net/samples/readings.csv;managed_identity=system')
    with (format='csv', ignoreFirstRecord=true)

RawReadings | count
RawReadings | take 10
```

KQL basics

KQL is a pipe-based read-only query language: start with a table, then pass rows through operators with `|`. It is case-sensitive for names and operators.

Operator	Purpose	SQL equivalent
`where`	Filter rows	WHERE
`project` / `project-away` / `extend`	Choose / drop / add columns	SELECT list
`summarize ... by`	Aggregate	GROUP BY
`top n by` / `order by` / `take`	Sort, limit	ORDER BY, TOP
`join kind=...`	Join	JOIN
`union`	Combine tables	UNION ALL
`bin(ts, 5m)`	Time buckets	DATE_TRUNC
`make-series`	Regular time series	none
`parse` / `extract` / `parse_json`	Text and JSON parsing	string funcs, JSON_VALUE

Operator	Purpose	SQL equivalent
`mv-expand`	Explode arrays	CROSS APPLY OPENJSON
`let`	Variables, sub-queries	CTE

```
Table
| where ts > ago(1h) and col == "value"
| extend newcol = expr
| summarize agg = count() by key, bin(ts, 5m)
| order by agg desc
| take 100
```

KQL query examples

Each query below runs against the `kql_iot` database (`RawReadings`, `Devices`, `Clicks`).

1. Filter and project the last 15 minutes

```
RawReadings
| where ts > ago(15m) and store_id == 1042
| project ts, device_id, temp_c, humidity
| order by ts desc
```

2. Aggregate per 5-minute bucket

```
RawReadings
| where ts between (datetime(2026-10-26 00:00) .. datetime(2026-10-27 00:00))
| summarize avg_temp = avg(temp_c), max_temp = max(temp_c), readings = count()
    by store_id, bin(ts, 5m)
```

3. Top N with `top` and percentiles

```
RawReadings
| where ts > ago(1d)
| summarize p50 = percentile(temp_c, 50), p95 = percentile(temp_c, 95) by device_id
| top 10 by p95 desc
```

4. Join with a dimension table (lookup is cheaper for small right sides)

```
RawReadings
| where ts > ago(1h)
| lookup kind=leftouter (Devices | project device_id, device_type, store_name) on device_id
| summarize avg(temp_c) by store_name, device_type
```

5. Inner join between two event tables

```
let orders = Clicks | where event == "purchase" and ts > ago(1d) | project session_id, order_ts =
ts;
let carts  = Clicks | where event == "add_to_cart" and ts > ago(1d) | project session_id, cart_ts =
ts;
carts
| join kind=inner orders on session_id
| extend minutes_to_buy = datetime_diff('minute', order_ts, cart_ts)
| summarize avg(minutes_to_buy), count()
```

6. Time series chart

```
RawReadings
| where ts > ago(7d) and device_id == "fridge-0412"
| summarize avg(temp_c) by bin(ts, 1h)
| render timechart
```

7. `make-series` and anomaly detection

```
RawReadings
| where ts > ago(14d)
| make-series avg_temp = avg(temp_c) default = 0 on ts step 1h by device_id
| extend anomalies = series_decompose_anomalies(avg_temp, 2.5)
| mv-expand ts to typeof(datetime), avg_temp to typeof(real), anomalies to typeof(int)
| where anomalies != 0
```

8. Parse JSON payloads

```
Clicks
| where ts > ago(1h)
| extend payload = parse_json(raw)
| project ts, session_id, page = tostring(payload.page),
          referrer = tostring(payload.ctx.referrer), items = toint(payload.cart.items)
```

9. Explode arrays with `mv-expand`

```
Clicks
| where event == "purchase"
| extend lines = parse_json(raw).lines
| mv-expand line = lines
| summarize units = sum(toint(line.qty)) by sku = tostring(line.sku)
| top 20 by units
```

10. Text parsing with `parse` and `extract`

```
AppLogs
| where message has "Payment failed"
| parse message with * "order=" order_id:long " code=" error_code:string " " *
| extend gateway = extract(@"gw=(\w+)", 1, message)
| summarize failures = count() by error_code, gateway
```

11. `arg_max` to get the latest row per key (dedupe)

```
RawReadings
| summarize arg_max(ts, *) by device_id
| project device_id, last_seen = ts, temp_c
| where last_seen < ago(10m)          // devices that went silent
```

12. Sessionization and funnels with `row_window_session` / `funnel_sequence`

```
Clicks
| where ts > ago(1d)
| order by user_id asc, ts asc
| extend session_start = row_window_session(ts, 30m, 30m, user_id != prev(user_id))
| summarize pages = count(), duration = max(ts) - min(ts) by user_id, session_start
```

13. Cross-database and cross-source queries

```
union
  (database("kql_iot").RawReadings | where ts > ago(1h) | extend src = "iot"),
  (database("kql_archive").RawReadings | where ts > ago(1h) | extend src = "archive")
| summarize count() by src
```

14. Query OneLake / lakehouse data via shortcut (external table)

```
external_table("lh_gold_dim_store")       // OneLake shortcut created in the KQL DB
| join kind=inner (RawReadings | where ts > ago(1h) | summarize avg(temp_c) by store_id) on
store_id
```

15. Stored function

```
.create-or-alter function with (docstring = "Hot fridges in last N minutes")
HotDevices(window:timespan = 15m, threshold:real = 8.0) {
    RawReadings
    | where ts > ago(window)
    | summarize avg_temp = avg(temp_c) by device_id, store_id
    | where avg_temp > threshold
}
// usage
HotDevices(30m, 7.5)
```

16. T-SQL against KQL (subset, for SQL users)

```
SELECT TOP 10 device_id, AVG(temp_c) AS avg_temp
FROM RawReadings
WHERE ts > DATEADD(hour, -1, GETUTCDATE())
GROUP BY device_id ORDER BY avg_temp DESC;
-- explain shows the KQL translation:  EXPLAIN SELECT ...
```

Update policies

An update policy runs a KQL function on each new batch ingested into a source table and appends the result to a target table, inside the same ingestion. Use it to parse raw JSON into typed tables (bronze to silver inside the Eventhouse). Mark it transactional so a failure in the policy fails the ingestion and nothing is lost silently.

```
.alter table Target policy update
@'[{"IsEnabled": true, "Source": "SourceTable", "Query": "FunctionName()", "IsTransactional": true,
"PropagateIngestionProperties": false}]'
```

Example: raw JSON events into a typed table.

```
.create table RawEvents (raw: dynamic, ingest_ts: datetime)
.create table Readings (device_id: string, store_id: int, temp_c: real, ts: datetime)

.create-or-alter function ParseReadings() {
    RawEvents
    | extend r = raw
    | project device_id = tostring(r.device_id), store_id = toint(r.store_id),
              temp_c = toreal(r.temp_c), ts = todatetime(r.ts)
    | where isnotempty(device_id) and isnotnull(ts)
}

.alter table Readings policy update
@'[{"IsEnabled": true, "Source": "RawEvents", "Query": "ParseReadings()", "IsTransactional":
true}]'

.alter table RawEvents policy retention '{"SoftDeletePeriod": "3.00:00:00"}'   // keep raw briefly
```

Materialized views

A materialized view keeps an aggregation over a source table up to date as data arrives, and queries combine the materialized part with the not-yet-processed delta, so results are always current. Typical uses: latest record per key (`arg_max`) and pre-aggregated rollups. Create with `backfill=true` to include existing data.

```
.create [async] materialized-view [with (backfill=true)] ViewName on table Source
{ Source | summarize ... by ... }
```

Example: device last-known state and an hourly rollup.

```
.create materialized-view with (backfill=true) DeviceLatest on table Readings
{
    Readings | summarize arg_max(ts, *) by device_id
}

.create async materialized-view with (backfill=true) ReadingsHourly on table Readings
{
    Readings | summarize avg_temp = avg(temp_c), max_temp = max(temp_c), n = count()
             by store_id, device_id, hour = bin(ts, 1h)
}

DeviceLatest | where ts < ago(10m)                          // silent devices, instantly
materialized_view("ReadingsHourly") | where hour > ago(7d)  // materialized part only (fastest)
```

Trap: update policies transform *each ingested batch* (row-level ETL); materialized views maintain *aggregations*. Interviewers like to ask which to use: parse/clean = update policy; dedupe or rollup = materialized view.

Management commands for operations

```
.show tables details
.show table Readings extents | summarize sum(OriginalSize), sum(ExtentSize)
.show ingestion failures | where FailedOn > ago(1d)
.show materialized-view ReadingsHourly          // check IsHealthy, MaterializedTo
.show queries | where StartedOn > ago(1h) | top 10 by Duration desc
```

Example: daily health check query for an on-call runbook.

```
.show ingestion failures
| where FailedOn > ago(24h)
| summarize failures = count(), sample_error = any(Details) by Table, ErrorCode
| order by failures desc
```

Real-Time dashboards and Activator

Real-Time dashboards

Real-Time dashboards are built on KQL queries with parameters, auto refresh (down to seconds) and cross-filtering. Each tile is a KQL query; dashboard parameters are referenced as variables.

```
// Tile "Avg temp by store", parameters _startTime, _endTime, _store (multi-select)
Readings
| where ts between (_startTime .. _endTime)
| where isempty(_store) or store_id in (_store)
| summarize avg(temp_c) by store_id, bin(ts, 5m)
| render timechart
```

Example settings: auto refresh 30 seconds, minimum refresh interval set by the dashboard editor, data source `kql_iot`, shared with store operations as Viewer.

```
Dashboard > Manage > Auto refresh: Enabled, default 30s, minimum 10s
```

Activator

Activator watches data (an Eventstream, a KQL query, a Power BI visual, Fabric events) and fires actions when a rule's condition is met: email, Teams message, run a pipeline/notebook/Spark job, Power Automate

flow. Rules are defined on **objects** (for example each device) with **properties** and **conditions** such as "becomes greater than", "stays above for 5 minutes", "no data for 10 minutes". Since 2026, Monitor hub job alerts are powered by Activator too.

```
act_alerts > Object: Fridge (key: device_id)
  Property: temp_c (from es_telemetry)
  Rule "Fridge too warm": temp_c > 8 AND stays for 5 minutes
  Action: Teams message to channel "Store Ops" + run pipeline pl_open_ticket (param device_id)
```

Example: a KQL-based Activator rule source (query runs every 5 minutes).

```
Readings
| where ts > ago(5m)
| summarize avg_temp = avg(temp_c) by device_id, store_id
| where avg_temp > 8
```

Power BI semantic models and Direct Lake

Storage modes

Import, DirectQuery and Direct Lake

A semantic model stores tables in one of several modes. **Direct Lake** reads Delta tables from OneLake straight into the VertiPaq engine on demand (no scheduled import, no copy), so reports get import-like speed on data that is as fresh as the last Delta commit.

Mode	Data location	Freshness	Gotcha
Import	Copy in model	Scheduled refresh	Refresh time and memory
DirectQuery	Source	Real time	Slower; every visual queries source
Direct Lake on SQL endpoint	OneLake via SQL endpoint metadata	Near real time	Falls back to DirectQuery for views, RLS on endpoint, or guardrail breaches
Direct Lake on OneLake	OneLake Delta directly	Near real time	No DirectQuery fallback; can mix tables from several lakehouses; uses OneLake security
Composite	Mix	Mixed	Relationship limits across modes

```
lh_gold > New semantic model > select tables (gold.fact_sales, gold.dim_*) > "sm_sales"
Model settings > Direct Lake behavior: Automatic | Direct Lake only | DirectQuery only
```

Example: decide the mode for Contoso.

```
fact_sales 1.8 B rows, updated hourly by Spark  → Direct Lake (V-Order on, OPTIMIZE nightly)
budget table from Excel, 2,000 rows             → Import (small, changes monthly)
live stock levels from SQL database             → DirectQuery (needs second-level freshness)
```

Framing, guardrails and fallback

Framing is the point where the model takes a snapshot of the Delta table versions it reads; with "Keep your Direct Lake data up to date" on, the model reframes automatically when tables change. **Guardrails** per SKU cap the rows per table, files and row groups per table, and model size in memory. Exceeding them in Direct Lake on SQL causes fallback to DirectQuery (slower); Direct Lake on OneLake fails the query instead.

```
Guardrails (F64 example; check current docs): max rows per table ~1.5 B, max model memory 25 GB,
max Parquet files per table 5,000, max row groups per table 5,000
```

Example: avoid fallback by compacting files and checking with DAX Studio / a trace.

```
spark.sql("OPTIMIZE lh_gold.gold.fact_sales VORDER")   # fewer files and row groups
```

```
Semantic model > Refresh (framing) ; Performance analyzer shows "DirectQuery" events if fallback
occurs
```

DAX basics for engineers

Measures and calculated columns

DAX measures are evaluated at query time in the filter context of the visual. Prefer measures over calculated columns (calculated columns are not supported in Direct Lake tables; push them to the gold table instead).

```
Measure Name = <aggregation or expression>
CALCULATE(<expr>, <filter1>, <filter2>)
```

Example: core sales measures.

```
Revenue        = SUM ( fact_sales[net_amount] )
Orders         = DISTINCTCOUNT ( fact_sales[order_id] )
Avg Order Value = DIVIDE ( [Revenue], [Orders] )
Revenue LY     = CALCULATE ( [Revenue], SAMEPERIODLASTYEAR ( dim_date[date] ) )
Revenue YoY %  = DIVIDE ( [Revenue] - [Revenue LY], [Revenue LY] )
Online Revenue = CALCULATE ( [Revenue], dim_channel[channel] = "Online" )
Revenue YTD    = TOTALYTD ( [Revenue], dim_date[date] )
```

Querying a model from a notebook (semantic link)

Semantic link (`sempy`) lets engineers read model tables and evaluate measures from Python, which is handy for testing measures against warehouse numbers.

```
import sempy.fabric as fabric
df = fabric.evaluate_measure("sm_sales", measure=["Revenue", "Orders"],
                             groupby_columns=["dim_store[region]"])
```

Example: reconcile the model with the gold table after each load.

```
model = fabric.evaluate_measure("sm_sales", ["Revenue"],
["dim_date[date]"]).set_index("dim_date[date]")
gold = spark.sql("SELECT order_date, SUM(net_amount) rev FROM lh_gold.gold.fact_sales "
                 "WHERE order_date >= '2026-10-01' GROUP BY
order_date").toPandas().set_index("order_date")
diff = model.join(gold, how="inner")
assert (abs(diff["Revenue"] - diff["rev"]) < 0.01).all(), "Model and gold disagree"
```

Model refresh from a pipeline or API

Import models need refresh; Direct Lake models only need reframing (automatic if enabled). Pipelines have a Semantic model refresh activity; the enhanced refresh REST API supports table-level refresh.

```
POST https://api.powerbi.com/v1.0/myorg/groups/{wsId}/datasets/{datasetId}/refreshes
{ "type": "full", "commitMode": "transactional", "objects": [ { "table": "budget" } ] }
```

Example: refresh only the Import budget table after the Excel load lands, from a notebook.

```
import sempy.fabric as fabric
fabric.refresh_dataset("sm_sales", objects=[{"table": "budget"}], refresh_type="full")
```

SQL database in Fabric and mirroring

SQL database in Fabric

What it is

SQL database in Fabric is an operational (OLTP) database built on the Azure SQL Database engine, provisioned as a Fabric item in seconds. It reached GA at Ignite in November 2025. Data is automatically replicated in near real time to OneLake as Delta, with a SQL analytics endpoint, so analytics never hit the OLTP engine. Engineers use it for app back ends, metadata/control tables for pipelines (watermarks, configs) and reverse-ETL targets.

```
CREATE TABLE ops.load_config (
  config_id INT IDENTITY(1,1) PRIMARY KEY,
  source_schema SYSNAME NOT NULL, source_table SYSNAME NOT NULL,
  target_table NVARCHAR(256) NOT NULL, watermark_column SYSNAME NULL,
  load_type VARCHAR(20) NOT NULL CHECK (load_type IN ('full','incremental','cdc')),
  enabled BIT NOT NULL DEFAULT 1);
```

Example: metadata-driven ingestion with a config table (full SQL Server features: enforced PKs, nvarchar, triggers, indexes).

```
INSERT INTO ops.load_config (source_schema, source_table, target_table, watermark_column,
load_type)
VALUES ('dbo','orders','bronze.orders','modified_at','incremental'),
       ('dbo','customers','bronze.customers','modified_at','incremental'),
       ('dbo','stores','bronze.stores',NULL,'full');
-- Pipeline Lookup: SELECT * FROM ops.load_config WHERE enabled = 1;
```

SQL database vs Warehouse vs Lakehouse

Need	Choose
Many small transactions, enforced keys, app back end	SQL database
Large analytical T-SQL transforms, star schema, multi-table transactions	Warehouse
Spark, files, data science, semi-structured data	Lakehouse
Telemetry, logs, time series, sub-second ingestion	Eventhouse

```
Rule of thumb: rows per write small + latency ms → SQL database
               rows per write millions + scans → Warehouse / Lakehouse
```

Example: a store-ops app writes stock adjustments to `sqldb_storeops`; the replica in OneLake feeds `lh_silver` through a shortcut, so the nightly gold build reads them without any pipeline.

```
-- SQL analytics endpoint of the SQL database (read-only replica in OneLake)
SELECT store_id, sku, SUM(qty_delta) AS adj FROM dbo.stock_adjustments
WHERE adjusted_at >= '2026-10-26' GROUP BY store_id, sku;
```

Mirrored databases

Database mirroring setup and monitoring

Mirroring takes an initial snapshot and then applies changes continuously. Each mirrored table shows status, rows replicated and last sync time. Not all data types or table shapes are supported (for example, tables

without primary keys or some LOB types may be skipped depending on source), so read the source's limitations page.

```
Mirrored database > Monitor replication
  Table            Status      Rows replicated   Last completed
  dbo.orders       Running     48,201,337        2026-10-27 09:14:55
  dbo.audit_log    Not supported (column type)   -
```

Example: start/stop mirroring and check status by REST in a runbook.

```
POST https://api.fabric.microsoft.com/v1/workspaces/{wsId}/mirroredDatabases/{id}/startMirroring
POST
https://api.fabric.microsoft.com/v1/workspaces/{wsId}/mirroredDatabases/{id}/getTablesMirroringStat
us
```

Open mirroring

Open mirroring lets any application write change data into a mirrored database landing zone. You create a folder per table with a `_metadata.json` declaring key columns, then drop sequentially numbered Parquet files with a `__rowMarker__` column (0 = insert, 1 = update, 2 = delete, 4 = upsert). Fabric merges them into Delta tables.

```
// LandingZone/customers/_metadata.json
{ "keyColumns": ["customer_id"] }
```

Example: a Python producer writing an upsert batch.

```
import pandas as pd
batch = pd.DataFrame({
  "customer_id": [101, 102, 103],
  "email": ["a@contoso.com", "b@contoso.com", None],
  "__rowMarker__": [4, 4, 2]})         # upsert, upsert, delete
batch.to_parquet("/tmp/00000000000000000042.parquet", index=False)
# upload to:
https://onelake.dfs.fabric.microsoft.com/<ws>/<mirror>.MountedRelationalDatabase/Files/LandingZone/
customers/
```

Mirroring Azure Databricks Unity Catalog

Metadata mirroring of a Databricks Unity Catalog catalog creates shortcuts to Databricks-managed Delta tables, so Fabric (SQL endpoint, Direct Lake, Spark) can read them without copying. Writes stay in Databricks.

```
New item > Mirrored Azure Databricks catalog > connection to workspace URL
  > choose catalog "retail_prod" > schemas/tables (auto-sync new tables: on)
```

Example: Direct Lake report over a Databricks gold table.

```
SELECT TOP 10 * FROM retail_prod_mirror.gold.fact_sales ORDER BY order_date DESC;
```

Security and governance

Access model layers

Layer	Controls	Example
Tenant / admin portal	Who can create items, export, share, use features	Disable "Users can create Fabric items" except for sg-fabric-creators
Capacity	Who can assign workspaces	Capacity contributors
Workspace roles	Everything in a workspace	Contributor on ws-retail-dev
Item permissions	One item (Read, ReadData, ReadAll, Build, Write, Reshare, Execute)	Share lh_gold with ReadData only
Compute / engine security	T-SQL GRANT, RLS, CLS, DDM; semantic model RLS/OLS	RLS on fact_sales
OneLake security	Roles on tables/folders with row/column rules, enforced across engines	StoreManagers role on lh_gold
Network	Private links, managed private endpoints, trusted workspace access, outbound protection	Tenant private link
Information protection	Sensitivity labels, DLP, Purview	"Confidential" label inherited downstream

Workspace roles

The four roles

Capability	Admin	Member	Contributor	Viewer
Manage workspace settings, delete workspace	Yes	No	No	No
Add members / manage roles	Yes (all)	Yes (Member and below)	No	No
Share items, manage item permissions	Yes	Yes	No	No
Create, edit, delete items; run notebooks/pipelines	Yes	Yes	Yes	No
Read data via Spark / OneLake files	Yes	Yes	Yes	No (unless OneLake security role or ReadAll)
Read via SQL endpoint / view reports	Yes	Yes	Yes	Yes (subject to SQL/OneLake security)

```
POST https://api.fabric.microsoft.com/v1/workspaces/{wsId}/roleAssignments
{ "principal": { "id": "<entra group object id>", "type": "Group" }, "role": "Contributor" }
```

Example: Contoso assignments.

```
ws-retail-prod:  Admin = sg-fabric-platform   Member = sp-fabric-deploy (service principal)
                 Contributor = none (changes only via deployment)   Viewer = sg-sales-analysts
ws-retail-dev:   Contributor = sg-data-engineers
```

Trap: Contributor and above bypass OneLake security roles and can read every file in the workspace. Give consumers Viewer or item-level sharing, never Contributor, if you rely on row or column rules.

Item permissions

Sharing items

Sharing a lakehouse or warehouse grants Read (connect, see metadata) plus optional extras: **ReadData** (query via SQL endpoint, subject to SQL security), **ReadAll** (read all files via Spark/OneLake), **Build** (build reports on the default semantic model). Pipelines and notebooks have Read/Write/Execute style permissions through workspace roles.

```
lh_gold > Share > sg-finance-analysts
  [x] Read all SQL endpoint data (ReadData)
  [ ] Read all Apache Spark and subscribe to events (ReadAll)
  [ ] Build reports on the default semantic model
```

Example: a finance analyst needs SQL access to gold only. Share `lh_gold` with ReadData, add a SQL role with GRANT on the `finance` schema, and don't give a workspace role.

```
CREATE ROLE finance_readers;
ALTER ROLE finance_readers ADD MEMBER [sg-finance-analysts@contoso.com];
GRANT SELECT ON SCHEMA::finance TO finance_readers;
```

OneLake security

Roles, row and column rules

OneLake security (GA spring 2026; the successor and superset of OneLake data access roles) defines roles inside a lakehouse (and other items) that grant Read (and optionally ReadWrite) on specific tables or folders, with optional row-level (SQL predicate) and column-level constraints. Rules are enforced in OneLake, so Spark, the SQL analytics endpoint (in user identity mode), Direct Lake on OneLake and shortcuts all honor them. Each lakehouse starts with a `DefaultReader` role that gives ReadAll holders access to everything; edit or delete it when you introduce restrictive roles.

```
lh_gold > Manage OneLake security > + New role "StoreManagersEU"
  Permission: Read   Data: Tables/gold/fact_sales, Tables/gold/dim_store
  Row security (fact_sales):  SELECT * FROM gold.fact_sales WHERE region = 'EU'
  Column security (dim_store): hide manager_salary
  Members: sg-store-managers-eu (Entra group) or virtual members "users with ReadData"
```

Example: the same role through the data access security REST API (shape abbreviated).

```
PUT https://api.fabric.microsoft.com/v1/workspaces/{wsId}/items/{lhId}/dataAccessRoles
{ "value": [ {
  "name": "StoreManagersEU",
  "decisionRules": [ { "effect": "Permit", "permission": [
      { "attributeName": "Path", "attributeValueIncludedIn": ["Tables/gold/fact_sales",
"Tables/gold/dim_store"] },
      { "attributeName": "Action", "attributeValueIncludedIn": ["Read"] } ] } ],
  "members": { "microsoftEntraMembers": [ { "tenantId": "...", "objectId": "<sg-store-managers-eu>"
} ] }
} ] }
```

> **Best practice:** define security once in OneLake security for lakehouse data, set the SQL analytics endpoint to user identity mode, and use Direct Lake on OneLake so the same rules flow into reports. Keep warehouse T-SQL security for warehouse tables.

Information protection and Purview

Sensitivity labels, DLP and Purview

Microsoft Purview sensitivity labels (Confidential, Highly Confidential) can be applied to Fabric items manually, by default label policies or by inheritance (downstream items inherit the label of upstream sources). Labels travel with exports to Excel/PDF. Purview DLP policies scan supported items (semantic models, lakehouses and others) for sensitive data. The Purview hub in Fabric gives admins reports; Purview Unified Catalog scans Fabric for lineage and classification. Audit logs record activities.

```
Admin portal > Information protection > [x] Allow users to apply sensitivity labels
  [x] Apply sensitivity labels from data sources to their data in Power BI
  [x] Automatically apply sensitivity labels to downstream content
Item > Settings > Sensitivity label: Confidential\Finance
```

Example: set a label on a lakehouse with the admin REST API (bulk).

```
POST https://api.fabric.microsoft.com/v1/admin/items/bulkSetLabels
{ "items": [ { "id": "<lh_gold id>", "type": "Lakehouse" } ],
  "labelId": "d9f2c3a1-....-confidential-finance" }
```

Lineage and impact analysis

Workspace lineage view shows item-to-item dependencies (source → pipeline → lakehouse → model → report). Impact analysis lists downstream items and owners before you change a table.

```
Workspace > View: Lineage > select lh_silver > Impact analysis
  → 3 notebooks, 2 semantic models, 11 reports, 214 viewers in 4 workspaces > Notify contacts
```

Example: before dropping column `legacy_flag`, run impact analysis on `lh_silver`, notify owners of the 2 models, then drop it in dev and promote.

```
ALTER TABLE lh_silver.customer.customers SET TBLPROPERTIES ('delta.columnMapping.mode'='name');
ALTER TABLE lh_silver.customer.customers DROP COLUMN legacy_flag;
```

Network security

Inbound: private links and conditional access

Tenant-level private link blocks public internet access to the whole tenant (all traffic via private endpoints in your VNet). **Workspace-level private link** restricts inbound access to specific workspaces. **Entra Conditional Access** can require compliant devices or named locations.

```
Admin portal > Tenant settings > Advanced networking > Azure Private Link: On
  [x] Block public Internet access
Azure: create Microsoft.PowerBI/privateLinkServicesForPowerBI resource + private endpoint + DNS
zones
```

Example: a regulated workspace with workspace private link enabled; users connect only from the corporate VNet, and the public endpoint returns 403 for that workspace.

```
Workspace settings > Inbound networking > Workspace-level private link: Private endpoints only
```

Outbound: managed private endpoints, trusted workspace access, gateways

Feature	Direction	Purpose
Managed private endpoints	Fabric Spark → Azure PaaS	Spark reaches firewalled SQL, storage, Key Vault privately

Feature	Direction	Purpose
Managed VNet	Spark	Created automatically when you add managed private endpoints
Trusted workspace access	Shortcuts/pipelines/COPY → firewalled ADLS	Storage account resource instance rule trusts a workspace identity
VNet data gateway	Dataflows, semantic models, pipelines → VNet sources	No VM to maintain
On-premises data gateway	→ on-prem sources	Classic gateway
Outbound access protection	Workspace → anywhere	Block outbound connections except approved ones (also for OneLake shortcuts)

```
Workspace settings > Network security > Managed private endpoints > + Create
  Name: mpe-sql-erp   Resource ID: /subscriptions/.../Microsoft.Sql/servers/sql-erp   Sub-resource: sqlServer
  → approve the pending connection on the Azure SQL server's Networking blade
```

Example: trusted workspace access for an ADLS account with "public network access disabled except trusted".

```
"networkAcls": {
  "resourceAccessRules": [ {
    "tenantId": "<tenant>",
    "resourceId": "/subscriptions/00000000-0000-0000-0000-000000000000/resourcegroups/Fabric/providers/Microsoft.Fabric/workspaces/<workspaceId>"
  } ],
  "defaultAction": "Deny"
}
```

Workspace identity and service principals

A **workspace identity** is an automatically managed Entra identity tied to a workspace, used to authenticate shortcuts, pipelines and connections to firewalled resources without secrets. **Service principals** (and managed identities) can call Fabric REST APIs and own CI/CD; tenant setting "Service principals can use Fabric APIs" must allow them.

```
Workspace settings > Workspace identity > + Workspace identity   (creates Entra app "ws-retail-prod")
Storage account IAM > Storage Blob Data Reader > assign to the workspace identity
Connection > Authentication kind: Workspace identity
```

Example: a service principal authenticating in a CI job.

```
from azure.identity import ClientSecretCredential
cred = ClientSecretCredential(tenant_id=TENANT, client_id=SP_ID, client_secret=SP_SECRET)
token = cred.get_token("https://api.fabric.microsoft.com/.default").token
```

Encryption and customer-managed keys

OneLake data is encrypted at rest with Microsoft-managed keys by default. Customer-managed keys (CMK) in Azure Key Vault can be applied at workspace level, and cluster-level CMK support reached GA in September 2026.

```
Workspace settings > Encryption > Customer-managed key: On > Key Vault URI / key name / version
```

Example: compliance requires key revocation to make data unreadable. Store the key in a Key Vault with purge protection; revoking access blocks reads of the encrypted workspace items.

```
az keyvault key create --vault-name kv-retail-cmk --name fabric-ws-prod --kty RSA --size 4096
az keyvault set-policy --name kv-retail-cmk --object-id <Fabric platform principal> --key-
permissions get wrapKey unwrapKey
```

CI/CD and automation

Git integration

Connecting a workspace to Git

Git integration syncs a workspace with a branch in Azure DevOps or GitHub (cloud, and GitHub Enterprise). Supported items are serialized as folders of definition files (notebooks as `.py`, pipelines as JSON, semantic models as TMDL/PBIP, warehouses as SQL projects). You commit from the workspace and update the workspace from Git. Only item definitions go to Git, never data.

```
Workspace settings > Git integration > Provider: Azure DevOps
  Organization: contoso  Project: retail-data  Repo: fabric-retail  Branch: main  Folder:
/workspace
Source control pane: Changes (commit) | Updates (pull) | Branches (branch out to new workspace)
```

Example: repository layout after the first commit.

```
/workspace
  lh_silver.Lakehouse/        .platform  lakehouse.metadata.json  shortcuts.metadata.json
  nb_silver_orders.Notebook/  .platform  notebook-content.py
  pl_daily_sales.DataPipeline/ .platform pipeline-content.json
  wh_sales.Warehouse/         .platform  wh_sales.sqlproj  sales/Tables/fact_sales.sql
  sm_sales.SemanticModel/     .platform  definition/ (TMDL)
  vl_retail.VariableLibrary/  variables.json  valueSets/Test.json  valueSets/Prod.json
```

Branching strategies

Strategy	How	Good for
Workspace per developer (branch out)	"Branch out to new workspace" creates a feature workspace on a feature branch	Teams of several engineers
Shared dev workspace on `main`	Everyone commits in one workspace	Solo or very small team
Git-based deploy (each stage workspace connected to its own branch)	PR from `dev` to `test` to `main` updates stage workspaces	Teams preferring Git flow over deployment pipelines

```
git checkout -b feature/orders-scd2 && git push -u origin feature/orders-scd2
# In Fabric: Source control > Branches > Branch out to new workspace "ws-retail-feat-scd2"
```

Example: the feature workflow.

```
1 Branch out → ws-retail-feat-scd2 (on capacity cap-retail-dev-f8)
2 Develop, run notebooks against dev lakehouses (Variable library default set)
3 Commit in workspace → PR feature/orders-scd2 → main (code review, build validation)
4 Merge → ws-retail-dev updates from Git → deployment pipeline promotes to test/prod
```

Deployment pipelines

Stages, rules and deploys

Deployment pipelines copy item definitions between workspaces assigned to stages (2 to 10 stages, typically Dev, Test, Prod). They compare stages, deploy selected items, and use **deployment rules** (data source and parameter rules) or **Variable library value sets** so each stage points to its own sources. Data is not copied.

```
Deployment pipelines > + New "dp-retail" > stages Development | Test | Production
Assign ws-retail-dev, ws-retail-test, ws-retail-prod
Compare → Deploy selected items (pl_daily_sales, nb_*, lh_* schema, sm_sales) → add note
Test stage > Deployment rules > sm_sales > Data source rule → lh_gold (test)
```

Example: deploy with the REST API from a release pipeline.

```
POST https://api.fabric.microsoft.com/v1/deploymentPipelines/{pipelineId}/deploy
{
  "sourceStageId": "<dev stage id>", "targetStageId": "<test stage id>",
  "items": [ { "sourceItemId": "<pl_daily_sales id>", "itemType": "DataPipeline" },
             { "sourceItemId": "<nb_silver_orders id>", "itemType": "Notebook" } ],
  "note": "Release 2026.10.27 - SCD2 for dim_customer",
  "options": { "allowOverwriteArtifact": true, "allowCreateArtifact": true }
}
```

Trap: deploying a lakehouse deploys the item (and shortcuts), not its tables or files. Warehouse deployments apply schema changes; review them in Compare because dropping a column drops data.

fabric-cicd and code-first deployment

The fabric-cicd library

`fabric-cicd` is an open-source Python library (officially supported by Microsoft since February 2026) that deploys item definitions from a Git folder to a workspace, with a `parameter.yml` to replace IDs, connection strings and values per environment. It deploys all in-scope items each time (not diffs) and can remove orphaned items. It suits Azure DevOps or GitHub Actions pipelines where you don't want Fabric deployment pipelines.

```
pip install fabric-cicd
```

```
from fabric_cicd import FabricWorkspace, publish_all_items, unpublish_all_orphan_items
ws = FabricWorkspace(
    workspace_id="<prod workspace id>",
    environment="PROD",                      # selects values in parameter.yml
    repository_directory="./workspace",
    item_type_in_scope=["Notebook", "DataPipeline", "Environment", "Lakehouse", "SemanticModel",
"VariableLibrary"],
)
publish_all_items(ws)
unpublish_all_orphan_items(ws)
```

Example: `parameter.yml` and a GitHub Actions job.

```
# parameter.yml
find_replace:
  - find_value: "7d1c-dev-lakehouse-guid"          # dev lh_silver id inside notebooks
    replace_value:
      TEST: "7d1c-test-lakehouse-guid"
      PROD: "7d1c-prod-lakehouse-guid"
  - find_value: "sql-dev.contoso.com"
    replace_value: { TEST: "sql-test.contoso.com", PROD: "sql-prod.contoso.com" }
```

```yaml
# .github/workflows/deploy-prod.yml
on: { push: { branches: [ main ] } }
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: prod
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install fabric-cicd azure-identity
      - run: python deploy.py --env PROD
        env:
          AZURE_CLIENT_ID: ${{ secrets.FABRIC_SP_ID }}
          AZURE_TENANT_ID: ${{ secrets.TENANT_ID }}
          AZURE_CLIENT_SECRET: ${{ secrets.FABRIC_SP_SECRET }}
```

Fabric REST APIs and the Fabric CLI

The Fabric REST API (`https://api.fabric.microsoft.com/v1/`) covers workspaces, items and definitions, jobs (run on demand, schedules), Git operations, deployment pipelines, OneLake shortcuts, admin APIs and more. Long-running operations return `202 Accepted` with a `Location` and `Retry-After` header to poll. The Fabric CLI (`fab`) wraps the API with a file-system-like interface.

```
pip install ms-fabric-cli
fab auth login
fab ls ws-retail-prod.Workspace
fab job run ws-retail-prod.Workspace/nb_silver_orders.Notebook -P run_date:string=2026-10-26
fab cp ws-retail-dev.Workspace/nb_silver_orders.Notebook ws-retail-test.Workspace/
```

Example: get an item definition, then update it in another workspace (the core of any custom deployer).

```python
import base64, json, requests, time
def lro(resp, H):
    while resp.status_code == 202:
        time.sleep(int(resp.headers.get("Retry-After", 5)))
        resp = requests.get(resp.headers["Location"], headers=H)
    return resp
r = lro(requests.post(f"{API}/workspaces/{DEV}/items/{NB}/getDefinition", headers=H), H)
defn = r.json() if "definition" in r.json() else requests.get(r.headers["Location"] + "/result",
headers=H).json()
requests.post(f"{API}/workspaces/{TEST}/items/{NB_TEST}/updateDefinition", headers=H, json=defn)
```

Testing data pipelines

Treat notebooks as code: keep transformation logic in functions (in a wheel or `%run` notebook), unit-test them with pytest on small DataFrames, and add data-quality checks (row counts, null checks, referential checks) as pipeline gates.

```python
# tests/test_transforms.py   (runs in CI with local pyspark)
from contoso_etl.transforms import clean_orders
def test_clean_orders_dedupes(spark):
    df = spark.createDataFrame([(1, "2026-10-26", "new"), (1, "2026-10-26", "NEW ")],
                               "order_id long, order_date string, order_status string")
    out = clean_orders(df)
    assert out.count() == 1 and out.first().order_status == "NEW"
```

Example: a DQ gate notebook that fails the pipeline.

```
checks = {
  "no_null_keys": spark.sql("SELECT COUNT(*) c FROM lh_silver.sales.orders WHERE order_id IS
NULL").first().c == 0,
  "fk_customers": spark.sql("""SELECT COUNT(*) c FROM lh_silver.sales.orders o
                               LEFT ANTI JOIN lh_silver.customer.customers c USING
(customer_id)""").first().c == 0,
}
failed = [k for k, ok in checks.items() if not ok]
if failed:
    raise Exception(f"DQ failed: {failed}")     # notebook activity fails → pipeline failure path
notebookutils.notebook.exit("DQ passed")
```

Monitoring and operations

Monitoring hub

Monitor hub and job history

The Monitor hub lists runs of pipelines, notebooks, Spark job definitions, dataflows, semantic model refreshes, Copy jobs and more across workspaces you can access, with filters by status, item type, time and submitter. Cross-workspace monitoring and Activator-powered job alerts were added in 2026, so you can alert on failures without building your own logging.

```
Monitor (left rail) > Filter: Status = Failed, Item type = Notebook, Start time = last 24h
  > select run > Details: duration, submitter, Spark application ID > "Open Spark UI"
Job alerts > + New alert > Item: pl_daily_sales > Condition: Failed > Teams channel "Data On-Call"
```

Example: list recent job instances of a pipeline with REST for a custom dashboard.

```
curl -s -H "Authorization: Bearer $TOKEN" \
  "https://api.fabric.microsoft.com/v1/workspaces/$WS/items/$PL/jobs/instances" \
  | jq -r '.value[] | [.startTimeUtc, .status, .failureReason.message] | @tsv' | head
```

Spark monitoring

Each Spark application has a detail page (jobs, stages, executors, logs, resource usage chart) and the Spark UI and history server. The Spark advisor flags skew, data spill and slow tasks in the notebook. For log export, the Fabric Apache Spark diagnostic emitter sends logs and metrics to Log Analytics, Event Hubs or storage.

```
Environment > Spark properties:
  spark.synapse.diagnostic.emitters = LA
  spark.synapse.diagnostic.emitter.LA.type = AzureLogAnalytics
  spark.synapse.diagnostic.emitter.LA.categories = Log,EventLog,Metrics
  spark.synapse.diagnostic.emitter.LA.workspaceId = <id>
  spark.synapse.diagnostic.emitter.LA.secret.keyVault = kv-retail ; ...secretName = la-key
```

Example: find a skewed stage. In the Spark UI Stages tab, a stage where the max task duration is 40 minutes and the median is 20 seconds points to a skewed join key; check the key distribution.

```
(spark.table("lh_silver.sales.order_lines").groupBy("customer_id").count()
      .orderBy("count", ascending=False).show(5))   # customer_id -1 (unknown) has 300M rows
```

Workspace monitoring

Workspace monitoring creates a read-only Eventhouse in the workspace that collects logs and metrics from items (semantic model queries, Eventhouse queries and ingestions, GraphQL, Spark and others as coverage grows), queryable with KQL and retained for 30 days.

```
Workspace settings > Monitoring > + Add monitoring eventhouse  (creates "Monitoring Eventhouse")
```

Example: slowest DAX queries in the last day from workspace monitoring.

```
SemanticModelLogs
| where Timestamp > ago(1d) and OperationName == "QueryEnd"
| project Timestamp, ItemName, ExecutingUser, DurationMs, EventText
| top 20 by DurationMs desc
```

OneLake diagnostics and audit

OneLake diagnostics logs data-access events (who read or wrote which path) into a lakehouse you choose, which helps with audit and cost attribution. Fabric activities also appear in the Microsoft Purview audit log.

```
Workspace settings > OneLake > Diagnostics: On > destination lh_ops (Files/diagnostics)
```

Example: who read the payroll folder last week (after loading diagnostic JSON into a table).

```
SELECT caller_upn, operation, COUNT(*) AS events
FROM lh_ops.dbo.onelake_access_log
WHERE path LIKE '%/Tables/hr/payroll/%' AND event_time >= DATEADD(day, -7, GETUTCDATE())
GROUP BY caller_upn, operation ORDER BY events DESC;
```

Performance tuning

Spark and Delta tuning

Checklist

Symptom	Likely cause	Fix
Many small files, slow reads	Frequent small appends, streaming	Optimize write, scheduled OPTIMIZE, fewer partitions
Slow session start	Custom pool cold start, many libraries	Starter pool, custom live pool, published environment, high concurrency
One task runs forever	Data skew	AQE skew join (`spark.sql.adaptive.skewJoin.enabled`), salting, filter null/unknown keys
Disk spill	Too few shuffle partitions or small executors	AQE, increase partitions, larger nodes
Big table broadcast fails	Broadcast threshold too high	Lower threshold, broadcast only small dims
Python UDF slow	Row-by-row serialization	Built-in functions, pandas UDFs, native engine
MERGE slow	Full target scan	Partition/cluster pruning in ON clause, Z-ORDER on key, deletion vectors
Direct Lake slow / fallback	Too many files/row groups, no V-Order	OPTIMIZE VORDER, readHeavyForPBI profile

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 64 * 1024 * 1024)
spark.conf.set("spark.native.enabled", "true")
```

Example: salting a skewed join key.

```
from pyspark.sql import functions as F
N = 16
big = spark.table("lh_silver.sales.order_lines").withColumn("salt", (F.rand() * N).cast("int"))
small = (spark.table("lh_silver.customer.customers")
         .withColumn("salt", F.explode(F.array([F.lit(i) for i in range(N)]))))
joined = big.join(small, ["customer_id", "salt"]).drop("salt")
```

Caching and pruning

```
df.cache(); df.count()     # materialize once if reused many times in a notebook
spark.sql("SELECT ... WHERE order_date = '2026-10-26'")   # partition pruning
```

Example: read only needed columns and push filters down to Parquet.

```
fact = (spark.read.table("lh_gold.gold.fact_sales")
        .select("order_date", "store_id", "net_amount")        # column pruning
        .where("order_date >= '2026-10-01'"))                   # predicate pushdown / file
skipping
```

Warehouse and SQL tuning

Checklist

Area	Practice
Data types	Smallest correct types; `varchar(n)` not `varchar(8000)`; avoid `datetime2(7)`
Loads	COPY INTO / CTAS in large batches; avoid singleton INSERTs (each creates small Parquet files)
Statistics	Let auto stats work; create FULLSCAN stats after big loads on join/filter columns
Queries	Filter early, avoid `SELECT *`, avoid scalar UDFs in hot paths
Isolation	Custom SQL pools for ETL vs BI; result set caching for repeated dashboard queries
Compaction	Warehouse compacts automatically; avoid many tiny transactions

```
-- Bad: 1 row per statement in a loop. Good: stage and load set-based.
INSERT INTO sales.fact_sales SELECT * FROM sales.stg_fact_sales WHERE load_id = 20261027;
```

Example: check a slow query's data scanned and decide whether better filters or stats are needed.

```
SELECT TOP 5 distributed_statement_id, total_elapsed_time_ms, data_scanned_remote_storage_mb,
       data_scanned_memory_mb, data_scanned_disk_mb, LEFT(command,120) AS q
FROM queryinsights.exec_requests_history
WHERE start_time > DATEADD(hour,-2,SYSUTCDATETIME())
ORDER BY data_scanned_remote_storage_mb DESC;
```

KQL and capacity tuning

KQL query hygiene

Filter on time first, use `has` over `contains` (term index vs substring scan), project early, put the smaller table on the left of `join` (or use `lookup`), and use materialized views for repeated aggregates.

```
// slower
Clicks | where page contains "checkout" | where ts > ago(1d)
// faster
Clicks | where ts > ago(1d) | where page has "checkout" | project ts, user_id, page
```

Example: `.show queries` to find the heaviest queries and rewrite them.

```
.show queries
| where StartedOn > ago(1d)
| project StartedOn, User, Duration, TotalCpu, MemoryPeak, Text
| top 10 by TotalCpu desc
```

Capacity cost tuning

Lever	Effect
Separate capacities for dev and prod	Dev experiments can't throttle prod
Autoscale billing for Spark	Spark off the shared capacity
Pause dev capacity outside working hours	Pay only when used
Reserved capacity for steady base load	Cheaper per CU than PAYG
Python notebooks for small data	2 VCores instead of a Spark cluster
Eventhouse: no minimum consumption in dev	Lets compute scale to zero-ish
Dataflow Gen2 → Copy job + notebook for big loads	Lower CU per GB

Example: monthly estimate for moving a 10-notebook nightly job from Spark to Python notebooks where data is under 1 GB each.

```
Before: 10 runs x 6 min x 16 VCores (starter pool min) = 960 VCore-min = 480 CU-min
After:  10 runs x 4 min x 2 VCores                     = 80 VCore-min  = 40 CU-min
Saving ~92% of CU for that job.
```

Fabric compared with other platforms

Platform comparison

Fabric vs Databricks vs Synapse vs Snowflake

Dimension	Microsoft Fabric	Azure Databricks	Azure Synapse Analytics	Snowflake
Model	SaaS, capacity (F-SKU)	PaaS, DBUs + your cloud VMs (serverless options)	PaaS, per-service (dedicated DWU, serverless per TB, Spark pools)	SaaS, credits per warehouse-second
Storage format	Delta in OneLake (Iceberg interop)	Delta (Unity Catalog, Iceberg via UniForm)	Proprietary (dedicated), Parquet/Delta (serverless/Spark)	Proprietary micro-partitions, Iceberg tables
Spark	Yes, native engine, starter pools	Yes, Photon, most mature	Yes (older runtimes)	Snowpark (not Spark)
SQL warehouse	Fabric Warehouse (T-SQL)	Databricks SQL	Dedicated / serverless SQL pools	Core strength
Streaming / logs	Eventstream + Eventhouse (KQL)	Structured Streaming, DLT/Lakeflow	Data Explorer pools (retired path)	Snowpipe Streaming, dynamic tables
BI	Power BI built in, Direct Lake	Partner BI, AI/BI dashboards	Power BI linked	Partner BI, Streamlit
Governance	OneLake catalog, OneLake security, Purview	Unity Catalog	Purview	Horizon
Strengths	One bill and one lake for BI + engineering, Microsoft 365 / Power BI integration, low ops	Advanced Spark/ML, fine-grained cluster control, multi-cloud	Mature dedicated SQL; existing estates	SQL simplicity, multi-cloud, data sharing
Watch outs	Shared capacity contention, feature maturity varies by item	Cost management, more to operate	Microsoft directs new investment to Fabric; migration paths exist	Spark-style workloads, egress for Power BI

```
Decision sketch:
  Power BI-centric Microsoft shop, small platform team        → Fabric
  Heavy ML/Spark engineering, multi-cloud, fine cluster tuning → Databricks (can mirror UC into
Fabric)
  SQL-first analytics across clouds, data sharing marketplace  → Snowflake (Iceberg interop with
OneLake)
  Existing Synapse dedicated pool                               → plan migration to Fabric Warehouse
```

Example: migrating a Synapse dedicated SQL pool. Fabric offers a migration assistant for Synapse dedicated pools (and, in preview from September 2026, Teradata) that converts DDL, flags unsupported T-SQL, and copies data.

```sql
-- Synapse dedicated (before)
CREATE TABLE dbo.fact_sales (...) WITH (DISTRIBUTION = HASH(customer_id), CLUSTERED COLUMNSTORE
INDEX);
-- Fabric Warehouse (after): no distribution or index options
CREATE TABLE dbo.fact_sales (...);
```

Concept mapping for migrations

From	Fabric equivalent
ADF pipeline / Synapse pipeline	Fabric pipeline (import/upgrade tools exist); linked services → connections
ADF mapping data flow	Dataflow Gen2 or notebook
Databricks notebook / job	Fabric notebook / Spark job definition, pipeline schedule
Unity Catalog	OneLake catalog + OneLake security (+ UC mirroring)
Databricks DLT / Lakeflow declarative pipelines	Materialized lake views
ADX cluster	Eventhouse
Synapse serverless SQL over lake	Lakehouse SQL analytics endpoint, OPENROWSET
AAS / Power BI Premium P-SKU	Semantic models on F-SKU

```
Synapse workspace → Fabric: Spark pools → environments + custom pools; notebooks → import
                            Lake database → lakehouse + shortcuts to existing ADLS (no copy)
```

Example: zero-copy first step of a lake migration with shortcuts, so Fabric engines read existing ADLS Delta tables while jobs move over gradually.

```
lh_legacy > Tables > New shortcut > ADLS Gen2 >
https://contosodatalake.dfs.core.windows.net/curated/
  select folders sales_orders, customers (Delta) → appear as tables, queryable via SQL endpoint
```

Interview Q&A: Fabric data engineer

Forty questions that come up in Fabric data engineering interviews, grouped roughly as fundamentals, design scenarios, troubleshooting and performance, and experience-based questions. Answers give the what, the why and the trade-off; follow-ups are what a good panel asks next.

Fundamentals

Q1. What is OneLake and why does it matter for a data engineer?

OneLake is one logical ADLS Gen2-based lake per tenant, created automatically, where every Fabric item stores its data as Delta/Parquet. It matters because every engine (Spark, T-SQL, KQL, Analysis Services) reads the same copy, so you stop building copy pipelines between a lake, a warehouse and BI. Shortcuts extend it to ADLS, S3, GCS and other workspaces without moving data. *Follow-up:* how do you address a OneLake table from outside Fabric?
(`abfss://<ws>@onelake.dfs.fabric.microsoft.com/<item>.Lakehouse/Tables/...`, Entra auth, ADLS SDKs.)

Q2. Lakehouse or Warehouse: how do you choose?

Choose by team skills and write pattern. Lakehouse: Spark writes, files plus tables, semi-structured data, data science; the SQL endpoint is read-only. Warehouse: T-SQL writes, multi-table transactions, stored procedures, SQL-first teams. Both store Delta in OneLake and can query each other with three-part names, so many estates use a lakehouse for bronze/silver and a warehouse (or lakehouse) for gold. *Follow-up:* can Spark write to a warehouse table? (No; Spark can read it through OneLake. Write via T-SQL, pipelines or the Spark connector for Warehouse.)

Q3. Explain shortcuts vs mirroring vs copying.

A shortcut is a pointer: no data copied, reads go to the source each time (with optional cache for S3/GCS). Mirroring continuously replicates a database into OneLake Delta using the source's change feed, so you get a local, query-optimized copy with low latency and no pipeline. Copying (Copy job, pipelines) is for sources that are neither Delta-addressable nor mirrorable, or when you need transformation and history. *Follow-up:* what happens to permissions through a OneLake shortcut? (The caller's identity is checked at the target item.)

Q4. What is V-Order and when would you turn it off?

V-Order is a Fabric write-time optimization of Parquet (sorting, row-group layout, encoding) that speeds up reads by VertiPaq, SQL and Spark while staying valid Parquet. It costs write time, so turn it off for write-heavy bronze/silver tables and turn it on for gold tables read by Direct Lake. Resource profiles (`writeHeavy`, `readHeavyForPBI`) set it with related options.

```
spark.conf.set("spark.fabric.resourceProfile", "readHeavyForPBI")
```

Follow-up: does V-Order lock you into Fabric? (No, the files are standard Parquet.)

Q5. What are the workspace roles and what can a Viewer do with lakehouse data?

Admin, Member, Contributor, Viewer. Admins manage everything, Members can also share and add members, Contributors create and edit items and read all data, Viewers can view items and query the SQL endpoint (subject to SQL security) but cannot read files through Spark unless granted ReadAll or a OneLake security role. *Follow-up:* why does a Contributor bypass OneLake security, and how do you design around it? (Write access implies full data access; give consumers Viewer or item sharing.)

Q6. How does Fabric capacity billing work, and what are smoothing and throttling?

You buy CUs (F2 to F2048). Operations consume CU-seconds; interactive ones are smoothed over at least 5 minutes and background ones over 24 hours, so short bursts don't fail. If smoothed use exceeds capacity, carry-forward builds. Past 10 minutes of future capacity, interactive requests are delayed 20 seconds; past 60 minutes they're rejected; past 24 hours background jobs are rejected too. *Follow-up:* how do you recover from background rejection quickly? (Scale up temporarily, or pause/resume and pay the debt; then find the culprit in the Capacity Metrics app.)

Q7. Why does F64 matter in licensing conversations?

F64 is the size where users with a free license can view Power BI content in the capacity; below F64, viewers need Pro or PPU licenses. It maps to the old P1. For a large reader base, F64 can be cheaper than many Pro licenses. *Follow-up:* are Fabric items available on F2? (Yes, all workloads; only the free-viewer rule and some limits differ.)

Q8. What's the difference between a starter pool, a custom pool and high concurrency mode?

Starter pools are Microsoft-managed pre-warmed Medium-node clusters with near-instant start. Custom pools let you choose node size and autoscale but cold start in minutes, unless you use custom live pools (GA September 2026). High concurrency mode is orthogonal: it shares one Spark session among multiple notebooks (same user, lakehouse, environment), saving start time and CU. *Follow-up:* what breaks session sharing? (Different environment, default lakehouse, Spark config or user.)

Q9. What does the native execution engine do and when won't it help?

It runs Spark SQL/DataFrame operators in vectorized C++ (Gluten + Velox) instead of the JVM, giving large speed-ups on scans, joins and aggregations over Parquet/Delta with no code change. It doesn't speed up Python UDFs, RDD code or unsupported operators, which fall back to the JVM. Check the plan for `Transformer` operators to confirm. *Follow-up:* how do you enable it for one notebook only? (`%%configure` with `spark.native.enabled=true`.)

Q10. Explain the SQL analytics endpoint and its sync behavior.

Each lakehouse (and mirrored DB, SQL database) gets a read-only T-SQL endpoint over its Delta tables. A background process syncs Delta metadata into the endpoint, usually within seconds, but a query right after a Spark write can see stale metadata. For pipelines that write then read, call the refresh metadata API first. *Follow-up:* what objects can you create on the endpoint? (Views, functions, procedures, security; not tables.)

Q11. What is Direct Lake and what is fallback?

Direct Lake loads Delta columns from OneLake into VertiPaq on demand, giving import-like speed without scheduled import refresh. Direct Lake on the SQL endpoint falls back to DirectQuery when a query hits a view, SQL-endpoint security, or a SKU guardrail (too many rows, files or row groups); Direct Lake on OneLake has no fallback and errors instead. *Follow-up:* how do you keep a large fact under guardrails? (OPTIMIZE, V-Order, fewer partitions, aggregate tables.)

Q12. Update policy vs materialized view in an Eventhouse?

An update policy runs a function on every ingested batch of a source table and writes rows to a target table; use it for parsing, typing and filtering (row-level ETL). A materialized view maintains an aggregation (`summarize`) incrementally; use it for latest-per-key (`arg_max`) and rollups. Make update policies transactional so failures surface.

```
.create materialized-view DeviceLatest on table Readings { Readings | summarize arg_max(ts, *)
by device_id }
```

Follow-up: how would you dedupe in KQL? (Materialized view with `take_any` / `arg_max` by key.)

Design and scenarios

Q13. Design a medallion architecture in Fabric for 200 source tables from SQL Server on-prem.

Metadata-driven: a control table (in a Fabric SQL database) lists tables, load type and watermark. Ingest with Copy job or a pipeline ForEach through an on-premises data gateway into `lh_bronze` (append, with ingest metadata). If the source supports it, mirroring SQL Server is simpler still. Silver notebooks dedupe and MERGE by key; gold builds facts and SCD2 dimensions, served with Direct Lake. Separate workspaces per layer and stage, Git integration, Variable libraries for connections, and DQ gates between layers. *Follow-up:* how do you handle hard deletes? (CDC/change tracking, mirroring, or periodic full-key reconcile with `WHEN NOT MATCHED BY SOURCE`.)

Q14. How would you implement SCD Type 2 in a lakehouse?

Use a single Delta MERGE with a staged union: one copy of changed source rows keyed by the business key to close the current row (`is_current=false`, `valid_to=now`), and one copy with a NULL merge key to insert the new version. Compare a hash of tracked columns, handle NULLs with null-safe equality, and dedupe the source first.

```
MERGE INTO dim t USING (SELECT key AS mk, * FROM src UNION ALL SELECT NULL, s.* FROM src s JOIN
dim d ON ... AND d.is_current AND s.hash <> d.hash) u
ON t.key = u.mk AND t.is_current
WHEN MATCHED AND t.hash <> u.hash THEN UPDATE SET is_current = false, valid_to =
current_timestamp()
WHEN NOT MATCHED THEN INSERT ...
```

Follow-up: how do you handle late-arriving changes out of order? (Use source effective timestamps for valid_from/valid_to and process in order, or rebuild the affected key's history.)

Q15. Build an incremental load with a watermark. What are the pitfalls?

Lookup the old watermark, lookup the new max at the source, copy rows `> old AND <= new`, then update the watermark only on success. Pitfalls: taking `utcNow()` after the copy (misses in-flight rows), non-monotonic or unindexed columns, timezone mismatches, missing deletes, and updating the watermark when the copy failed. *Follow-up:* when would you switch to CDC? (Deletes matter, no reliable modified column, or high change rates.)

Q16. A business wants dashboards with under 10-second latency on IoT data plus 2 years of history for ML. Design it.

Eventstream from IoT Hub; route raw events to an Eventhouse table (direct ingestion) with an update policy to a typed table and materialized views for rollups; Real-Time dashboard on KQL with 10-30 s refresh; Activator for alerts. Enable OneLake availability on the typed table (or a second Eventstream destination to a lakehouse) for long history; Spark reads that Delta for ML. Retention in KQL 90 days hot, history in OneLake. *Follow-up:* how do you control Eventhouse cost? (Caching policy shorter than retention, no minimum consumption outside prod, materialized views instead of repeated big aggregates.)

Q17. How do you secure a gold lakehouse so store managers see only their region in Spark, SQL and Power BI?

Use OneLake security: a role per region (or one role with a predicate based on a mapping) with table access and a row filter; members are Entra groups. Put the SQL analytics endpoint in user identity mode and build the model on Direct Lake on OneLake so the same rules apply. Managers get Viewer or item sharing, never Contributor. *Follow-up:* what if the gold layer is a warehouse? (T-SQL RLS with a security policy, CLS via column GRANTs, and model RLS if using import.)

Q18. How would you organize workspaces and capacities for a 30-engineer data platform?

Workspaces per domain and layer with dev/test/prod, feature workspaces via branch-out. At least two capacities: a smaller one for dev/test (paused at night) and production on its own capacity, optionally with Spark on autoscale billing. Domains in the admin portal for delegation, Certified endorsement on gold items, and Variable libraries for per-stage settings. *Follow-up:* how do you stop one team's backfill from throttling finance reports? (Separate capacities, surge protection thresholds, autoscale billing for Spark, scheduling.)

Q19. Copy job, pipeline Copy activity, Dataflow Gen2 or notebook: walk through your choice.

Copy job for straightforward bulk/incremental/CDC copies of many tables without orchestration. Pipeline Copy when the copy is one step in a larger flow with dependencies and parameters. Dataflow Gen2 for Power Query transformations owned by analysts on moderate volumes. Notebooks for large volumes and complex logic. Compare CU usage in the Metrics app; Dataflow Gen2 tends to cost more per GB at scale. *Follow-up:* how do you call a Copy job from a pipeline? (The Copy job activity.)

Q20. How do you implement CI/CD for Fabric?

Git integration on dev (Azure DevOps or GitHub) with feature branches and PRs. Promote either with deployment pipelines (with deployment rules/Variable library value sets) or code-first with `fabric-cicd` from a CI pipeline using a service principal and `parameter.yml`. Add unit tests for transformation functions and DQ gates. Keep data out of Git, and handle warehouse schema changes carefully because deployment applies DDL. *Follow-up:* deployment pipelines vs fabric-cicd? (Pipelines: UI, compare, rules, approvals in Fabric. fabric-cicd: full code control, works with any CI system, deploys full state each time.)

Q21. When would you use materialized lake views instead of notebooks?

When the transformation is expressible in Spark SQL and is a chain of dependent tables: MLVs give declarative definitions, automatic dependency ordering, incremental refresh where possible, data-quality constraints (DROP or FAIL) and lineage. Notebooks remain better for procedural logic, API calls, ML, and complex SCD2 or custom error handling. *Follow-up:* status? (Spark SQL MLVs GA since March 2026; PySpark authoring preview.)

Q22. A source team wants to push changes into Fabric from a custom application. Options?

Open mirroring (write Parquet change files with `__rowMarker__` into the landing zone; Fabric merges), Eventstream custom endpoint (events, Kafka/Event Hubs protocol), OneLake ADLS APIs into `Files/` followed by a notebook, or writing to a Fabric SQL database that auto-replicates to OneLake. *Follow-up:* which gives you a SQL endpoint with no code on the Fabric side? (Open mirroring or SQL database.)

Q23. How do you migrate from Azure Synapse dedicated SQL pools?

Assess T-SQL compatibility (no distributions/indexes in Fabric Warehouse, some types differ), use the migration assistant to convert DDL and move data, rewrite unsupported constructs, rebuild pipelines in Fabric Data Factory, re-point Power BI to the warehouse or Direct Lake, run in parallel and reconcile numbers before cutover. For the lake side, shortcut existing ADLS Delta first and move jobs gradually. *Follow-up:* what disappears from your tuning toolkit? (Distribution keys, CCI maintenance, resource classes; replaced by automatic management and SQL pools.)

Q24. How would you share Fabric data with an external Snowflake or Databricks team?

Options: OneLake external data sharing (in-place, read-only sharing to another Fabric tenant), Iceberg metadata virtualization so Snowflake can read Fabric Delta as Iceberg, ADLS-compatible OneLake access for Databricks with a service principal, or mirroring Unity Catalog the other way round. Govern with sensitivity labels and OneLake security. *Follow-up:* what would you check before sharing? (Labels, PII columns, legal agreements, audit via OneLake diagnostics.)

Troubleshooting and performance

Q25. Reports got slow at 2 p.m. every day this week. How do you investigate?

Open the Capacity Metrics app, select the 2 p.m. timepoints, check utilization and throttling charts, then drill into background and interactive operations to find the top CU consumers. Usually a scheduled or ad hoc Spark/dataflow job overlaps with report peak. Fix by rescheduling, moving it to another capacity or autoscale billing, tuning the job, or setting surge protection. *Follow-up:* how is that job's cost still felt hours later? (Background smoothing over 24 hours.)

Q26. A MERGE into a 2 TB Delta table takes 50 minutes. What do you look at?

Is the source deduplicated and small? Does the ON clause allow file pruning (date or cluster key in the condition)? Is the target Z-ordered or clustered on the merge key, and are files compacted? Are deletion vectors enabled? Is the native engine on? Check the Spark UI for number of files touched and skewed tasks.

```
MERGE INTO t USING s ON t.order_id = s.order_id AND t.order_date >= '2026-10-01' ...
```

Follow-up: what if updates span all dates? (Cluster on the key, or split hot vs cold data.)

Q27. A table written by Spark doesn't appear in the SQL analytics endpoint. Why?

Common causes: it was saved under `Files/` instead of `Tables/` or as a non-Delta format; metadata sync hasn't run yet; unsupported column types or names; it's an external table outside `Tables/`; or a Delta feature the endpoint doesn't read. Check the lakehouse explorer for "Unidentified" folders and the endpoint's sync status. *Follow-up:* how do you force sync? (Refresh metadata API or the UI refresh button.)

Q28. A notebook takes 4 minutes to start every run. How do you fix it?

It's probably on a custom pool (cold start) or installing many libraries at session start. Use the starter pool or a custom live pool, publish an environment so libraries are pre-built, avoid `%pip` in production, and use high concurrency so pipeline notebooks share a session. *Follow-up:* when is a Python notebook the better fix? (Small data; no Spark start at all.)

Q29. Your Direct Lake report falls back to DirectQuery. What do you do?

Confirm with Performance analyzer or a trace. Check guardrails for the SKU (rows, files, row groups per table, memory), views or SQL-endpoint security on the tables, and whether the model is Direct Lake on SQL. Compact and V-Order the tables, replace views with materialized tables, or switch to Direct Lake on OneLake with OneLake security. Set the fallback behavior explicitly so it fails visibly in testing. *Follow-up:* how does framing affect freshness? (Model reads the table versions as of the last framing; auto-reframe keeps it current.)

Q30. A pipeline sometimes fails with capacity errors overnight. What's your approach?

Check the Metrics app for background rejection or surge protection events at those times. Add retries with backoff, spread the schedule, move heavy Spark to autoscale billing or another capacity, and fix the root cause (a job consuming most of the 24-hour budget). Alert on failures through Monitor hub job alerts. *Follow-up:* why can daytime usage cause overnight rejections? (24-hour smoothing carries debt forward.)

Q31. KQL queries over 90 days are slow. How do you speed them up?

Filter on the datetime column first, use `has` instead of `contains`, project only needed columns, keep the hot cache window covering the common query range, pre-aggregate with materialized views, and use `lookup` for small dimensions. Check `.show queries` for CPU and memory peaks. *Follow-up:* what does caching policy cost? (Hot cache uses SSD/compute; longer hot windows cost more.)

Q32. Many small files in silver after a streaming job. What's your plan?

Turn on optimize write, use a longer trigger interval or `availableNow` micro-batches, run OPTIMIZE on recent partitions on a schedule (or the table maintenance API), avoid over-partitioning, and VACUUM weekly with at least 7 days of retention. *Follow-up:* how do you avoid conflicts between OPTIMIZE and the stream? (Delta handles concurrent append + compaction; avoid OPTIMIZE on partitions being rewritten by MERGE at the same time.)

Q33. Someone overwrote a gold table by mistake. How do you recover?

For a lakehouse table: `DESCRIBE HISTORY`, then `RESTORE TABLE ... TO VERSION AS OF n` (if not vacuumed). For a warehouse: time travel query or `CREATE TABLE ... AS CLONE OF ... AT '<time>'`, or restore the warehouse to a restore point. Then find out why the deployment or job could overwrite prod (permissions, missing checks). *Follow-up:* how long can you go back? (Lakehouse: VACUUM retention, default 7 days, and log retention 30 days; warehouse time travel default 30 days.)

Q34. A Spark job has one stage where a single task runs for 40 minutes. Diagnose and fix.

Data skew: one key holds a large share of rows (often NULL or "unknown"). Confirm with a key count. Fix with AQE skew join handling, filtering or handling the hot key separately, salting the key, or broadcasting the small side. *Follow-up:* why not just add executors? (The skewed task still runs on one core.)

Experience and judgement

Q35. Tell me about a time you reduced Fabric costs.

Strong answers give numbers: for example, "Our F128 ran at 95%. The Metrics app showed three Dataflow Gen2 loads using 40% of daily CU. I replaced them with a Copy job and a notebook, moved small transforms to Python notebooks, and enabled autoscale billing for Spark. We dropped to F64 and saved roughly half the capacity cost with faster loads." Structure it as situation, analysis, action, measured result. *Follow-up:* what did you trade off? (Analyst-owned dataflows became engineer-owned code; documentation and handover.)

Q36. How do you decide whether a preview feature goes to production?

Check the support statement (previews have no SLA), the roadmap and known limitations, test with production-like volume, have a fallback (feature flag or alternate path), and get stakeholder sign-off on the risk. For example, use GA Spark SQL MLVs in prod but keep PySpark MLVs in dev until GA. *Follow-up:* how do you track status changes? (Monthly Fabric feature summaries, release plan, Fabric blog.)

Q37. How do you handle data quality in Fabric?

Layered: schema enforcement in Delta, constraints in MLVs (`ON MISMATCH DROP/FAIL`), DQ notebooks (null keys, referential integrity, row-count deltas, freshness) gating the pipeline, quarantine tables for rejected rows, and alerts through Activator or Monitor hub. Report DQ metrics in a small dashboard so data owners see them. *Follow-up:* what do you do with bad rows? (Quarantine with reason codes; never drop silently in silver.)

Q38. How do you make notebooks production-ready?

Parameter cells, no hard-coded IDs (Variable libraries), shared logic in a wheel or common notebook with unit tests, idempotent writes (MERGE/replaceWhere), logging of row counts and run IDs, exit values for the pipeline, retries at activity level, Git with PR reviews, and environments pinned to a runtime. *Follow-up:* what makes a write idempotent? (Re-running for the same date yields the same result; no duplicate appends.)

Q39. How would you explain Fabric vs Databricks to a CIO?

Fabric is SaaS with one capacity bill, one lake and Power BI built in, which suits Microsoft-centric organizations with small platform teams. Databricks offers deeper Spark/ML control, multi-cloud and a mature Unity Catalog, but more to operate and govern. They interoperate (Delta, UC mirroring, OneLake ADLS APIs), so the choice can be per workload rather than all or nothing. *Follow-up:* where does each one cost more? (Fabric: idle reserved capacity and contention; Databricks: unmanaged clusters and DBU sprawl.)

Q40. What changed in Fabric over the past year that affects how you build?

Runtime 2.0 GA (Spark 4.1, Delta 4.2, Python 3.13), OneLake security GA, materialized lake views GA, SQL database GA, Copy job with CDC and SCD2, custom live pools, warehouse custom SQL pools and snapshots, fabric-cicd official support, Eventstream SQL operator GA, more mirroring sources, and agentic features (data engineering agent preview). Practically: fewer custom pipelines, security defined once in OneLake, and code-first CI/CD. *Follow-up:* what did you have to fix when moving to Runtime 2.0? (Rebuilt wheels for Python 3.13, retested Scala 2.13 JARs.)

Cheat sheet

Platform map

Need	Item
Spark + files	Lakehouse
T-SQL writes	Warehouse
OLTP app	SQL database
Logs/IoT	Eventhouse
Orchestrate	Pipeline
Bulk/CDC copy	Copy job
Power Query	Dataflow Gen2
Zero-ETL DB	Mirroring
No-copy link	Shortcut
Alerts	Activator

OneLake paths

```
abfss://<ws>@onelake.dfs.fabric.microsoft.com/
  <item>.Lakehouse/Tables/<schema>/<table>
  <wsId>/<itemId>/Files/<path>
```

Capacity

- 1 CU = 2 Spark VCores; Spark burst 3x
- F64 = P1; free viewers from F64
- Interactive smooth ≥5 min; background 24 h
- Throttle: ≤10 min ok; 10-60 delay 20 s; 60 min-24 h reject interactive; >24 h reject all
- Pause = pay carry-forward now
- Autoscale billing for Spark = PAYG off-capacity

Workspace roles

	Adm	Mem	Con	View
Settings	✔			
Share/members	✔	✔		
Edit/run items	✔	✔	✔	
Spark/OneLake read	✔	✔	✔	role
SQL/report read	✔	✔	✔	✔

Item permissions

Read · ReadData (SQL) · ReadAll (Spark/files) · Build · Write · Reshare · Execute

Runtimes

RT	Spark	Delta	Py
1.3	3.5	3.2	3.11
2.0	4.1	4.2	3.13

Spark settings

```
spark.conf.set("spark.native.enabled","true")
spark.conf.set("spark.fabric.resourceProfile","readHeavyForPBI")
spark.conf.set("spark.databricks.delta.optimizeWrite.enabled","true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled","true")
spark.conf.set("spark.ms.autotune.enabled","true")
```

notebookutils

```
notebookutils.fs.ls("Files/landing")
notebookutils.notebook.run("nb", 600, {"d":"x"})
notebookutils.notebook.runMultiple(dag)
notebookutils.notebook.exit("42")
notebookutils.credentials.getSecret(kv, name)
notebookutils.variableLibrary.getLibrary("vl")
```

PySpark I/O

```
spark.read.table("lh.schema.t")
df.write.mode("append").saveAsTable("s.t")
.option("mergeSchema","true")
.option("replaceWhere","d='2026-10-26'")
.option("overwriteSchema","true")
F.broadcast(dim); df.dropDuplicates([k])
```

Delta commands

```
MERGE INTO t USING s ON t.k=s.k
 WHEN MATCHED THEN UPDATE SET *
 WHEN NOT MATCHED THEN INSERT *
 WHEN NOT MATCHED BY SOURCE THEN DELETE;
OPTIMIZE t [WHERE p] [ZORDER BY (c)]
[VORDER];
VACUUM t RETAIN 168 HOURS [DRY RUN];
DESCRIBE HISTORY t;  DESCRIBE DETAIL t;
SELECT * FROM t VERSION AS OF 12;
SELECT * FROM t TIMESTAMP AS OF '2026-10-25';
RESTORE TABLE t TO VERSION AS OF 12;
ALTER TABLE t SET TBLPROPERTIES
 (delta.enableChangeDataFeed=true);
SELECT * FROM table_changes('t', 5);
CREATE TABLE t (...) CLUSTER BY (c);
```

SCD quick rules

- SCD1: MERGE update in place
- SCD2: union src (key) + src (NULL key) → close + insert
- Hash tracked cols; dedupe source; `<=>` for NULLs

Materialized lake view

```
CREATE MATERIALIZED LAKE VIEW s.v
(CONSTRAINT c CHECK (x IS NOT NULL)
 ON MISMATCH DROP) AS SELECT ...;
REFRESH MATERIALIZED LAKE VIEW s.v;
```

Warehouse T-SQL

```
COPY INTO s.t FROM 'https://.../*.parquet'
 WITH (FILE_TYPE='PARQUET');
CREATE TABLE s.n AS SELECT ...;       -- CTAS
CREATE TABLE s.c AS CLONE OF s.t
 [AT '2026-10-26T23:00:00'];
SELECT ... OPTION (FOR TIMESTAMP AS OF
 '2026-10-01T06:00:00.000');
SELECT * FROM OPENROWSET(BULK 'url',
 FORMAT='csv', HEADER_ROW=TRUE) r;
id BIGINT IDENTITY  -- no seed, gaps
PK/FK/UNIQUE ... NOT ENFORCED
SELECT * FROM
queryinsights.exec_requests_history;
```

Warehouse security

```
GRANT SELECT ON SCHEMA::sales TO [grp];
GRANT SELECT ON t (c1,c2) TO [grp]; -- CLS
CREATE SECURITY POLICY p ADD FILTER
 PREDICATE sec.fn(store_id) ON s.t; -- RLS
ALTER TABLE t ALTER COLUMN email
 ADD MASKED WITH (FUNCTION='email()');
GRANT UNMASK TO [grp];
```

Pipeline expressions

```
@pipeline().parameters.p   @variables('v')
@activity('A').output.firstRow.col
@activity('Copy').output.rowsCopied
@item().table_name   @pipeline().RunId
@formatDateTime(utcNow(),'yyyy/MM/dd')
@addDays(utcNow(),-1)   @concat(a,b)
@pipeline().libraryVariables.name
```

Watermark pattern

1 Lookup old WM → 2 Lookup new MAX → 3 Copy `> old AND <= new` → 4 Update WM on success. Deletes need CDC.

Ingestion choice

Case	Tool
Many tables, incr/CDC	Copy job
Step in workflow	Copy activity
Analyst transforms	Dataflow Gen2
Big/complex	Notebook
DB replica	Mirroring
Events	Eventstream

KQL core

```
T | where ts > ago(1h) and k == "x"
  | project a, b | extend c = a * 2
  | summarize count(), avg(v) by k, bin(ts, 5m)
  | top 10 by count_ desc
T | summarize arg_max(ts, *) by id
T | lookup (Dim) on id
T | join kind=inner (U) on id
T | extend j = parse_json(raw)
  | mv-expand x = j.items
T | make-series avg(v) on ts step 1h by id
  | render timechart
has (term) > contains (substring)
```

KQL management

```
.create table T (id:string, ts:datetime)
.alter table T policy retention
 '{"SoftDeletePeriod":"365.00:00:00"}'
.alter table T policy caching hot = 30d
.alter table Tgt policy update
@'[{"Source":

"Raw","Query":"Fn()","IsTransactional":true}]'
.create materialized-view with
(backfill=true)
 MV on table T { T | summarize ... by ... }
.show ingestion failures
```

Direct Lake

- On SQL endpoint: falls back to DQ (views, SQL RLS, guardrails)
- On OneLake: no fallback, OneLake security
- No calc columns; V-Order + OPTIMIZE gold
- Framing = snapshot of Delta versions

OneLake security

- Roles: tables/folders + row + column rules
- Enforced in Spark, SQL (user identity), Direct Lake on OneLake
- Admin/Member/Contributor bypass
- Edit/delete DefaultReader role

Network

Tenant/workspace private link (in) · Managed private endpoints (Spark out) · Trusted workspace access (ADLS firewall) · VNet / on-prem gateway · Outbound access protection · CMK

CI/CD

- Git: ADO/GitHub; branch out to workspace
- Deployment pipelines: 2-10 stages, rules
- Variable library: value sets per stage
- `pip install fabric-cicd` → `publish_all_items(ws)`
- REST: `api.fabric.microsoft.com/v1` ; 202 + Location → poll
- CLI: `fab ls | fab job run | fab cp`

Monitoring

Monitor hub (runs, job alerts) · Capacity Metrics app (14 days CU) · Spark UI/advisor · queryinsights · workspace monitoring Eventhouse · `.show queries` · OneLake diagnostics

Perf one-liners

- Small files → optimize write + OPTIMIZE
- Skew → AQE, salt, isolate hot key
- Slow start → starter/live pool, env, HC mode
- Small data → Python notebook
- Warehouse → batch loads, stats, SQL pools
- KQL → time filter first, `has` , MVs

Limits & defaults

- VACUUM default 7 days; log 30 days
- Warehouse time travel 30 days
- Trial capacity = F64, 60 days
- Mirroring free storage 1 TB per CU
- queryinsights 30 days history
- Workspace monitoring 30 days